

**НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ
СІКОРСЬКОГО»**

**Навчально-науковий інститут атомної та теплової енергетики
Кафедра інженерії програмного забезпечення в енергетиці**

**Розрахунково-графічна робота
ПРОЄКТУВАННЯ ТА СПЕЦИФІКАЦІЯ
ПРОСТОЇ ІМПЕРАТИВНОЇ МОВИ ПРОГРАМУВАННЯ
ЗАГАЛЬНОГО ПРИЗНАЧЕННЯ
(з дисципліни «Основи розробки трансляторів»)
Мова програмування Krok
Рівень 2 Основний**

Учасники проєкту:

Степаненко Назар Юрійович
Аніщенко Артем Олександрович

Виконав: студент III курсу, групи ТВ-43
спеціальності 121 «Інженерія програмного забезпечення»

Степаненко Назар Юрійович

(Прізвище, ім'я, по батькові)

— — — — — (Підпис)

Перевірів: доцент, к.т.н. Стативка Ю.І.

(Посада, науковий ступінь, прізвище та ініціали)

— — — — — (Підпис)

Київ – 2026

ЗМІСТ

1 Вступ	6
1.1 Назва мови	6
1.2 Рівень складності та мова-прототип	6
1.3 Призначення та ключові особливості	7
1.4 Обробка програми	8
1.5 Нотація	9
1.5.1 Розширена нотація Бекуса — Наура	9
1.5.2 Граматика мови у канонічній формі	9
1.5.3 Синтаксичні діаграми	10
1.5.4 Представлення семантики	11
1.6 Відповідність вимогам завдання	11
2 Лексична структура	13
2.1 Алфавіт	13
2.2 Роздільники рядків та пробільні символи	19
2.3 Коментарі	19
2.4 Лексеми і токени	20
2.5 Ідентифікатори	21
2.6 Ключові слова	22
2.7 Літерали	23
2.7.1 Цілі літерали	24
2.7.2 Дійсні літерали	24
2.7.3 Рядкові літерали	25
2.7.4 Логічні літерали	26
2.8 Оператори та роздільники	27

2.9 Автоматична вставка крапки з комою	30
2.10 Таблиця токенів	31
3 Типи даних і змінні	33
3.1 Значення, змінні та категорії змінних	33
3.2 Класифікація типів	34
3.3 Користувацькі скалярні типи	35
3.4 Сконструйовані типи	35
3.4.1 Масиви	36
3.4.2 Структури	37
3.4.3 Параметри-вказівники	38
3.5 Еквівалентність та сумісність типів	38
3.6 Приведення типів	39
3.6.1 Неявне приведення	39
3.6.2 Явне приведення	40
4 Вирази	41
4.1 Класифікація виразів	41
4.2 Синтаксис виразів	41
4.3 Обчислення виразів	46
4.4 Оператори: пріоритет, асоціативність, типи операндів і результату	47
4.4.1 Арифметичні оператори	47
4.4.2 Оператори порівняння	48
4.4.3 Логічні оператори	49
4.4.4 Операції над рядками	49
4.5 L-значення	49

4.6 Константні вирази	50
5 Інструкції	51
5.1 Блок і список інструкцій	51
5.2 Присвоювання значення.....	52
5.3 Інструкція розгалуження.....	53
5.4 Інструкція повторення.....	53
5.5 Інструкції введення та виведення	55
5.6 Інструкція повернення з функції	57
6 Декларації	58
6.1 Оголошення змінних	58
6.2 Іменовані константи	59
6.3 Оголошення типів.....	60
6.4 Оголошення функцій.....	60
6.5 Області видимості	62
6.6 Час життя та керування пам'яттю	63
7 Структура програми	64
7.1 Файл програми, пакет та імпорт	64
7.2 Функція main і модель виконання	65
7.3 Помилки трансляції	65
7.4 Помилки часу виконання	67
8 Властивості граматики	68
8.1 Кількісні характеристики	68
8.2 Практичні трансформації: факторизація та заміна рекурсії ітерацією	68
8.3 Стандартні трансформації граматик.....	69

8.4 Автоматична перевірка граматики	70
8.5 Граматика та контекстні умови	72
9 Приклади програм	73
9.1 Базовий приклад	73
9.2 Типи та приведення	75
9.3 Розгалуження	75
9.4 Цикли.....	75
9.5 Функції та рекурсія.....	76
9.6 Передача параметрів за посиланням	77
9.7 Масиви та структури	77
9.8 Області видимості та затінення	79
9.9 Помилки часу виконання	79
9.10 Некоректні програми.....	80
Перелік посилань.....	81
10 Повна граматика мови.....	82

1 Вступ

Цей документ є специфікацією мови програмування **Krok** — формальним і повним описом її лексики, синтаксису, системи типів та семантики. Структуру документа побудовано за переліком елементів специфікації імперативної мови, наведеним у розд. 2.4 навчального посібника [1]: вступ, лексична структура, типи даних і змінні, вирази, інструкції, декларації, структура програми, повна граматики мови. Додатково наведено розділ про властивості розробленої граматики та розділ контрольних прикладів програм.

1.1 Назва мови

Мова має назву **Krok**; назва вимовляється як «крок» і записується в текстах латиницею. Назва є транслітерацією українського слова «крок» і пов'язана з мовою-прототипом: англійське *go* означає «іди», а крок — елементарна одиниця руху. Так само кожна інструкція програми є одним кроком алгоритму, а сама мова є «кроком» від повноцінної мови Go до простої навчальної мови, для якої в межах дисципліни буде розроблено транслятор.

Файли з вихідним кодом мовою Krok мають розширення `.krok`.

1.2 Рівень складності та мова-прототип

Мова Krok відповідає **Рівню 2 (Основний)**: це процедурно-модульна статично типізована мова з користувацькими типами. Об'єктно-орієнтовані концепції не підтримуються.

Мовою-прототипом обрано мову **Go** (https://go.dev/doc/effective_go). З мови Go запозичено синтаксис оголошень (`var`, `const`, `type`, `func`), структуру програми (`package main`, `import "fmt"`, функція `main`), інструкцію присвоєння (`=`) та оголошення з ініціалізацією (`:=`), інкремент і декремент (`++`, `--`), інструкції повторення `for` та розгалуження `if-else`, введення/виведення через пакет `fmt`, однорядкові коментарі `//`, правило автоматичної вставки крапки з комою, масиви, структури та вказівники як механізм передачі параметрів за посиланням.

Семантику елементів мови адаптовано до потреб навчального транслятора. Основні відмінності від мови-прототипу наведено в табл. 1.1.

Таблиця 1.1 – Основні відмінності мови Krok від мови-прототипу Go

Аспект	Go	Krok
Приведення <code>int</code> → <code>float</code>	лише явне	неявне розширення та явне приведення
Піднесення до степеня	немає оператора	оператор <code>**</code> , правоасоціативний
Переповнення <code>int</code>	циклічне (<code>wrap-around</code>)	помилка часу виконання
Типи <code>int</code> , <code>float</code> , <code>bool</code> , <code>string</code>	попередньо оголошені ідентифікатори	ключові слова
Вказівники	довільні значення, <code>nil</code>	лише типи параметрів функцій
Складені літерали	для масивів, зрізів, структур, мап	лише для масивів із явним типом <code>[N]T</code>
Інструкція <code>for</code>	чотири форми, <code>break</code> , <code>continue</code>	дві форми: з передумовою та трикомпонентна
<code>if</code> з ініціалізацією, <code>switch</code>	є	немає
Зрізи, мапи, методи, інтерфейси, горутини	є	немає
Пакети	довільні	лише <code>main</code> та вбудований <code>fmt</code>

Проект виконується спільно (виконавці — Степаненко Н. Ю. та Аніщенко А.). Відповідно до вимог до спільних проєктів кожен виконавець обирає власний варіант інструкцій повторення та розгалуження. У цій специфікації описано варіант: **інструкція `for` з передумовою та трикомпонентна інструкція `for`** (розд. 5.4) і **інструкція `if` з необов'язковими гілками `else if` та `else`** (розд. 5.3).

1.3 Призначення та ключові особливості

Krok — проста імперативна процедурна мова програмування загального призначення, статично і суворо типізована, призначена для запису обчислювальних алгоритмів, що взаємодіють із середовищем через стандартні потоки введення та виведення: числові розрахунки, обробка масивів, рекурсивні алгоритми, прості моделі даних на основі структур.

Ключові особливості мови:

- статична типізація: тип кожної змінної та кожного виразу відомий на етапі трансляції;
- сувора типізація: єдиним неявним перетворенням є розширення `int` → `float`, решта перетворень — явні;
- обов'язкове оголошення імен і нульові значення для неініціалізованих змінних;
- семантика значень для масивів і структур; передача за посиланням лише через явні параметри-вказівники;
- відсутність динамічного розподілу пам'яті та збирача сміття: уся пам'ять — статична область і стек викликів;
- перевірка меж масивів та контроль переповнення чисел під час виконання;
- граматики мови належить до класу LL(1), тому транслятор будується методом рекурсивного спуску з одним токеном попереднього перегляду.

1.4 Обробка програми

Програма мовою Krok подається на вхід транслятора (компілятора або інтерпретатора) у вигляді одного текстового файлу в кодуванні UTF-8. Трансляція складається з таких фаз:

1. Лексичний аналіз — окремий прохід, що перетворює потік символів (`character stream`) на потік токенів (`token stream`), вилучає коментарі та пробільні символи і вставляє неявні крапки з комою (розд. 2.9). Уся інформація, необхідна для побудови лексичного аналізатора, міститься в розд. 2.
2. Синтаксичний аналіз — перевірка відповідності потоку токенів граматиці мови та побудова дерева розбору (`parse tree`).
3. Семантичний аналіз — перевірка контекстних умов: оголошеність імен, області видимості, сумісність типів тощо.
4. Генерування коду або безпосереднє виконання.

Результат трансляції виконується системою часу виконання (run-time system), яка отримує вхідні дані зі стандартного потоку введення, записує результати у стандартний потік виведення, а повідомлення про помилки — у стандартний потік помилок.

1.5 Нотація

1.5.1 Розширена нотація Бекуса — Наура

Для опису лексики та синтаксису мови Krok використовується розширена нотація Бекуса — Наура (РБНФ) у варіанті Н. Вірта, описаному в розд. 2.2.2 посібника [1]. Метасимволи нотації наведено в табл. 1.2.

Таблиця 1.2 – Прийнята нотація РБНФ

Метасимвол	Значення
$A = \alpha$	нетермінал A визначається як α
$\alpha \mid \beta$	альтернатива: α або β
$[\alpha]$	0 або 1 екземпляр α
$\{ \alpha \}$	0 або більше екземплярів α
$(\alpha \mid \beta)$	групування
Abc	нетермінал (починається з великої літери)
abc	термінал — ключове слово (починається з малої літери)
'+', 'Scan'	термінал, записаний у лапках буквально
'\t', '\n', '\r'	термінали: символи табуляції, нового рядка та повернення каретки
? текст ?	спеціальна послідовність: неформально описана множина символів
(* текст *)	коментар до граматики

Правило починається з імені нетерміналу на початку рядка; рядки, що починаються з пробілів, продовжують попереднє правило. Обраний варіант РБНФ не використовує ані символу завершення правила, ані знака конкатенації: послідовність символів позначається їх записом поруч.

1.5.2 Граматика мови у канонічній формі

Формально граматика мови Krok є контекстно вільною граматикою $G = (N, T, P, S)$, де N — множина нетерміналів, T — множина терміналів, P — множина продукцій виду $\alpha \rightarrow \beta$, S — аксіома (початковий символ). Алфавітом

граматики є $V = N \cup T$, а алфавітом самої мови — множина терміналів T . Оскільки лексичний аналіз виконується окремим проходом, граматику поділено на дві частини:

- **синтаксична граMATика** з аксіомою `Program` описує, як із tokenів утворюються програми; її терміналами є токени мови (розд. 2.10);
- **лексична граMATика** з аксіомою `SourceText` описує, як із символів алфавіту утворюються лексеми; її терміналами є символи алфавіту (розд. 2.1).

Нетермінали `Ident`, `IntLit`, `FloatLit` та `StringLit` є в синтаксичній граматиці неподільними токенами; нетермінали `BoolLit`, `RelOp`, `AssignOp` та `IncDecOp` групують токени і використовуються в обох частинах. Кожне правило має рівно один нетермінал у лівій частині, отже граMATика мови є контекстно вільною (тип 2 за класифікацією Хомського).

Лексика мови, як і лексика мов програмування загалом (розд. 2.1 у [1]), належить до вузького класу **регулярних** мов: жодне лексичне правило не є самовкладеним, тому потік символів розпізнається детермінованим скінченним автоматом, а не магазинним. Обидві частини записано однією нотацією РБНФ в одному файлі, тобто формально вони утворюють єдину контекстно вільну граматику мови. Кількісні характеристики граматики та результати її перевірки наведено в розд. 8.

1.5.3 Синтаксичні діаграми

Для графічного представлення граматики використовуються синтаксичні діаграми Вірта. Відповідно до розд. 2.2.4 посібника [1] для кожного правила граматики побудовано окрему діаграму у формі орієнтованого графа: термінали зображено в овалах (прямокутниках із заокругленими кутами), нетермінали — у прямокутниках (курсивом), спеціальні послідовності — курсивом без рамки. Вхід і вихід діаграми орієнтовано зліва направо; ланцюжок належить мові, якщо він описує шлях від входу до виходу.

Діаграми згенеровано автоматично з файлу граматики `grammar/krok.ebnf` програмою `tools/gen_diagrams.py` (бібліотека `railroad-diagrams`), тому кожне правило має рівно одну діаграму з ідентичними назвами символів.

Ту саму граматику експортовано в нотації всіх трьох генераторів діаграм, згаданих у розд. 2.2.4 посібника [1] (програма `tools/export_visualizer.py`), тому будь-яке правило мови можна відобразити кожним із цих інструментів:

- `grammar/krok-visualizer.ebnf` — EBNF Visualizer [6];
- `grammar/krok-iso14977.ebnf` — EBNF 2 RailRoad (нотація ISO/IEC 14977);
- `grammar/krok-w3c.ebnf` — Railroad Diagram Generator (нотація W3C).

1.5.4 Представлення семантики

Семантику мови представлено **неформально** — природномовним текстом, який супроводжує опис синтаксичних конструкцій, з таблицями типів, операторів і помилок та з прикладами програм (розд. 9). Такий спосіб опису відповідає розд. 2.3.2 посібника [1] і практиці специфікацій мов Python, Go та Pascal; формальних моделей (операційної, денотаційної чи аксіоматичної семантики) ця специфікація не використовує.

Опис семантики кожної конструкції вказує, за яких умов її виконання **завершується нормально**, а за яких виникає помилка. Перелік помилок наведено в розд. 7.3 (виявляються транслятором) та 7.4 (виявляються під час виконання).

1.6 Відповідність вимогам завдання

Таблиця 1.3 – Відповідність специфікації вимогам завдання

Вимога	Конструкція мови Krok	Розділ
Оператор циклу	<code>for умова { }, for init; умова; post { }</code>	5.4
Оператор розгалуження	<code>if умова { } else if умова { } else { }</code>	5.3
Цілі та дійсні числа, + - * /	типи <code>int</code> , <code>float</code> ; оператори + - * / %	3.2, 4.4.1
Піднесення до степеня (правоасоціативне)	оператор <code>**</code>	4.4
Логічний тип	<code>bool</code> , <code>true</code> , <code>false</code> , <code>&&</code> <code> </code> <code>!</code>	3.2, 4.4.3

Вимога	Конструкція мови Krok	Розділ
Рядковий тип	string, рядкові літерали	3.2, 4.4.4
Іменовані константи	const	6.2
Порівняння	== != < <= > >=	4.4.2
Дужки у виразах	(Expression)	4.2
Однорядкові коментарі	// ...	2.3
Приведення int ↔ float	неявне розширення, float(x), int(x)	3.6.1, 3.6.2
Введення/виведення	fmt.Scan, fmt.Print, fmt.Println	5.5
Функції, рекурсія	func, return	6.4
Передача за значенням і за посиланням	параметри T та *T, &x, *p	3.4.3
Локальні та глобальні змінні	блоки, затінення	6.5
Користувачькі скалярні типи	type Celsius float	3.3
Масиви (багатовимірні, статична та динамічна ініціалізація)	[N]T, [N]T{...}	3.4.1
Структури (факультативно)	type P struct { ... }	3.4.2
Модульна структура (факультативно)	import "fmt"	7.1

2 Лексична структура

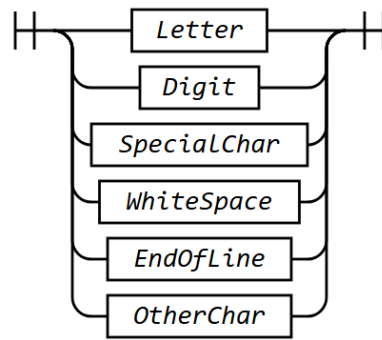
Лексичний аналіз виконується окремим проходом і не залежить від синтаксичних та семантичних аспектів мови. Лексичний аналізатор читає текст програми зліва направо і на кожному кроці виділяє **найдовшу** послідовність символів, що утворює лексему (правило найдовшого збігу). Наприклад, ****** розпізнається як один оператор піднесення до степеня, а не як два оператори множення; **<=** — як один оператор, а **a---b** — як **a**, **--**, **-**, **b**.

2.1 Алфавіт

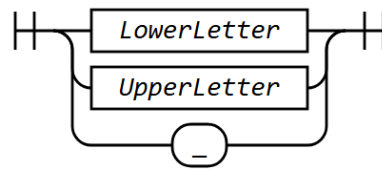
Текст програми мовою Krok записується в кодуванні UTF-8. Поза рядковими літералами та коментарями програма може містити лише латинські літери, цифри, спеціальні знаки, пробільні символи та символи кінця рядка. Усередині рядкових літералів і коментарів допускаються будь-які символи Unicode (зокрема кирилиця), крім символів кінця рядка. Мова чутлива до регістру літер: **total**, **Total** і **TOTAL** — різні ідентифікатори.

Alphabet	= Letter Digit SpecialChar WhiteSpace EndOfLine OtherChar									
Letter	= LowerLetter UpperLetter '_'									
LowerLetter	= 'a'	'b'	'c'	'd'	'e'	'f'	'g'	'h'	'i'	
	'j'	'k'	'l'	'm'	'n'	'o'	'p'	'q'	'r'	
	's'	't'	'u'	'v'	'w'	'x'	'y'	'z'		
UpperLetter	= 'A'	'B'	'C'	'D'	'E'	'F'	'G'	'H'	'I'	
	'J'	'K'	'L'	'M'	'N'	'O'	'P'	'Q'	'R'	
	'S'	'T'	'U'	'V'	'W'	'X'	'Y'	'Z'		
Digit	= '0'	NonZeroDigit								
NonZeroDigit	= '1'	'2'	'3'	'4'	'5'	'6'	'7'	'8'	'9'	
SpecialChar	= '+'	'-'	'*'	'/'	'%'	'='	'!'	'<'	'>'	
	'&'	' '	':'	';'	','	'.'	'('	')	'['	
	']'	'{'	'}'	'''	'\"					
OtherChar	= ? інший символ Unicode (лише в StringLit та Comment) ?									

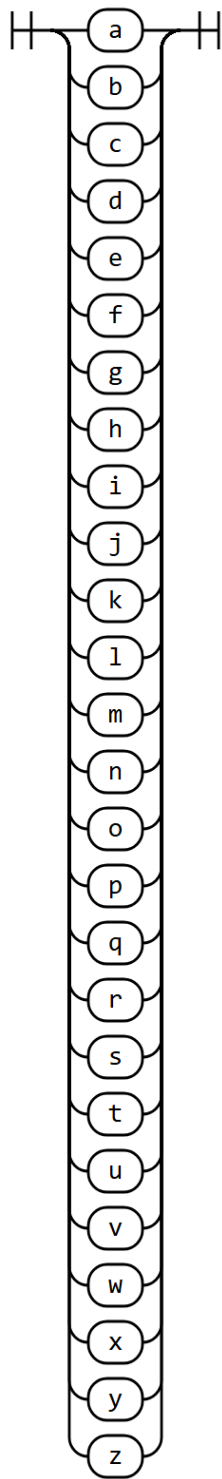
Alphabet



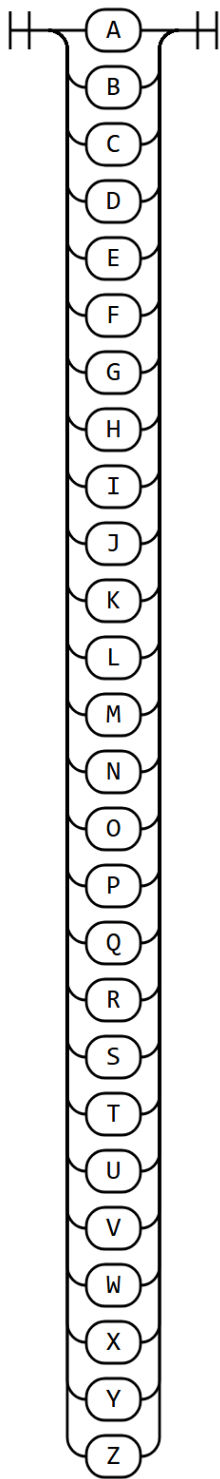
Letter



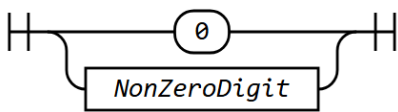
LowerLetter



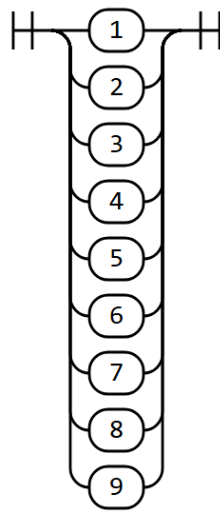
UpperLetter



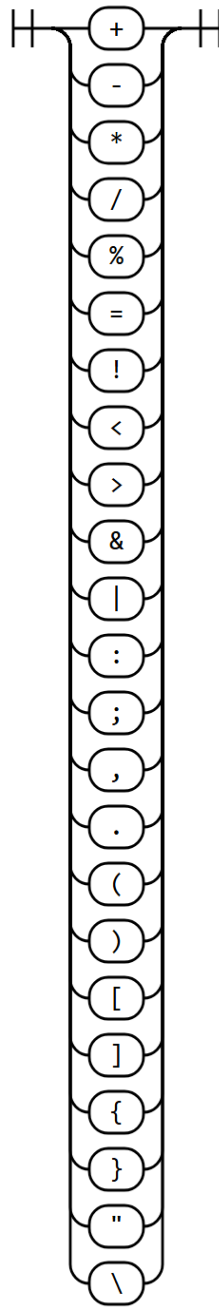
Digit



NonZeroDigit



SpecialChar



OtherChar

H ? *інший символ Unicode (лише в StringLit та Comment)* ? H

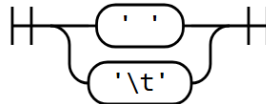
Рисунок 2.1 – Синтаксичні діаграми алфавіту

2.2 Роздільники рядків та пробільні символи

Пробільними символами є пробіл і горизонтальна табуляція. Роздільником рядків є символ нового рядка, якому може передувати символ повернення каретки (обидві форми, `'\n'` та `'\r' '\n'`, є допустимими).

```
WhiteSpace = ' ' | '\t'  
EndOfLine  = '\n' | '\r' '\n'
```

WhiteSpace



EndOfLine

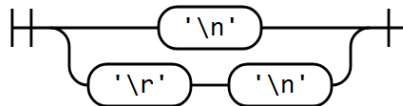


Рисунок 2.2 – Синтаксичні діаграми пробільних символів та роздільника рядків

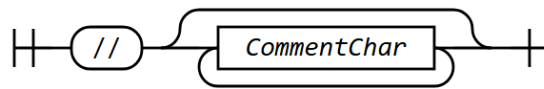
Пробільні символи та роздільники рядків не є токенами: вони лише розділяють лексеми і на цьому етапі вилучаються з потоку. Єдиний виняток — роздільник рядків, після якого за правилом розд. 2.9 може бути вставлено токен `;`. Кількість пробільних символів між токенами не обмежена і на зміст програми не впливає; відступи синтаксичного значення не мають.

2.3 Коментарі

Однорядковий коментар починається парою символів `//` поза рядковим літералом і продовжується до кінця рядка. Символ кінця рядка до коментаря не належить. Коментар вилучається лексичним аналізатором і діє як пробіл. Послідовність `//` усередині рядкового літерала коментарем не є. Багаторядкових коментарів мова не має.

```
Comment      = '//' { CommentChar }  
CommentChar  = ? будь-який символ Unicode, крім '\n' та '\r' ?
```

Comment



CommentChar

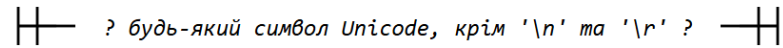


Рисунок 2.3 – Синтаксичні діаграми коментаря

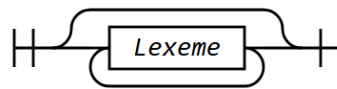
```
x := 10 // коментар до кінця рядка
s := "// це не коментар"
```

2.4 Лексеми і токени

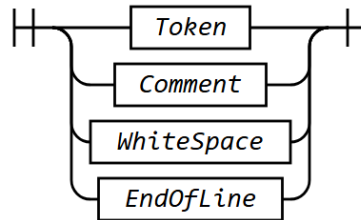
Текст програми є послідовністю лексем. Лексеми, що передаються синтаксичному аналізатору, називають токенами; це ідентифікатори, ключові слова, літерали, оператори та роздільники. Коментарі, пробільні символи та роздільники рядків токенами не є.

```
SourceText    = { Lexeme }
Lexeme        = Token | Comment | WhiteSpace | EndOfLine
Token         = Ident | Keyword | IntLit | FloatLit | StringLit | BoolLit
              | Operator | Delimiter
```

SourceText



Lexeme



Token

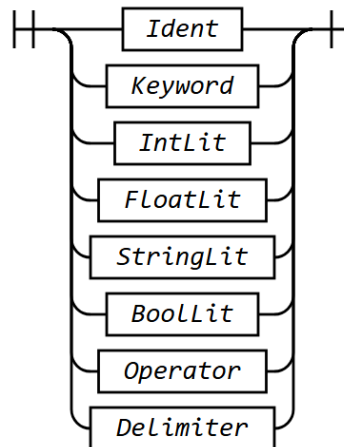


Рисунок 2.4 – Синтаксичні діаграми лексем

Між двома токенами, які інакше злилися б в один (наприклад, ідентифікатор і ключове слово), має бути принаймні один пробільний символ, роздільник рядків або коментар.

2.5 Ідентифікатори

Ідентифікатори позначають змінні, константи, типи, поля структур і функції. Ідентифікатор починається з літери або символу підкреслення, за якими можуть іти літери, підкреслення та цифри. Довжина ідентифікатора не обмежена, і всі його символи є значущими.

```
Ident      = Letter { Letter | Digit }
```

Ident

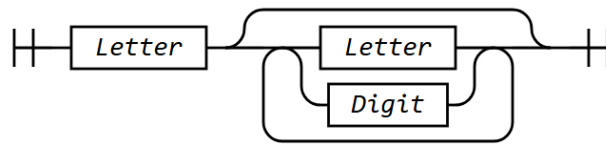


Рисунок 2.5 – Синтаксична діаграма ідентифікатора

Якщо лексема, що за формою могла б бути ідентифікатором, збігається з ключовим словом (розд. 2.6), вона є ключовим словом. Якщо вона збігається зі словом `true` або `false`, вона є логічним літералом.

Приклади: `x`, `total`, `Point`, `_tmp1`, `matrix2D`.

2.6 Ключові слова

Ключові слова зарезервовані й не можуть використовуватися як ідентифікатори.

Keyword	=	package		import		func		var		const		type		struct		
		if		else		for		return		int		float		bool		string
		len		fmt		'Print'		'Println'		'Scan'						

Keyword

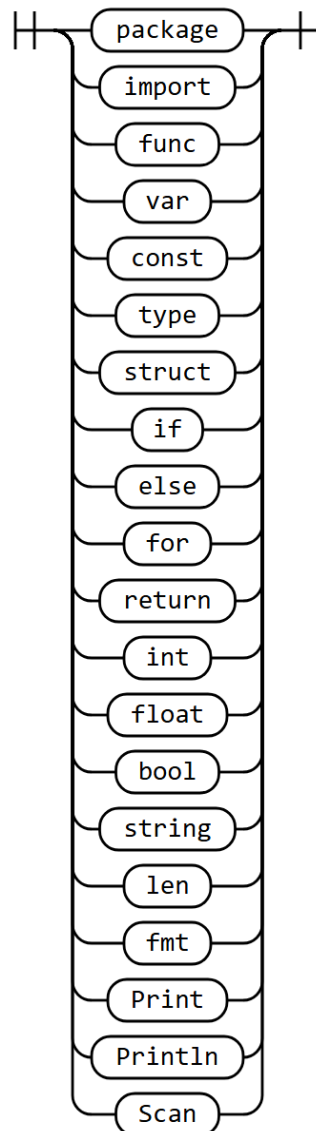


Рисунок 2.6 – Синтаксична діаграма ключових слів

Слова `int`, `float`, `bool` та `string` позначають базові типи, `len` — вбудовану функцію довжини. Слово `fmt` позначає вбудований пакет введення/виведення, а `Print`, `Println` і `Scan` — його операції. Слова `true` та `false` є логічними літералами (розд. 2.7).

2.7 Літерали

Літерали позначають сталі значення. Мова має літерали чотирьох видів: цілі, дійсні, рядкові та логічні; символічних літералів (окремого типу для одного символу) мова не передбачає. Тип літерала визначається його формою. Знак

числа до літерала не належить: у записі `-5` лексичний аналізатор виділяє два токени, оператор `-` і літерал `5`, а унарний мінус розпізнається під час синтаксичного аналізу.

2.7.1 Цілі літерали

Цілий літерал — десяткова послідовність цифр без знака. Щоб уникнути плутанини з вісімковими літералами мови Go, запис із ведучим нулем (наприклад, `007`) є лексичною помилкою; єдиний цілий літерал, що починається з `0`, — сам `0`. Цілий літерал має тип `int`, а його значення не може перевищувати `9223372036854775807`; інакше виникає помилка трансляції CE-10.

```
IntLit      = '0' | NonZeroDigit { Digit }
```

IntLit

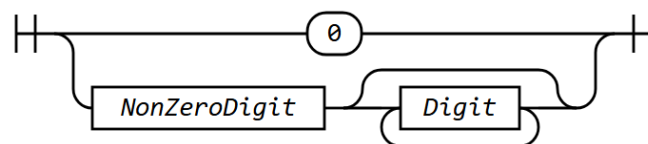


Рисунок 2.7 – Синтаксична діаграма цілого літерала

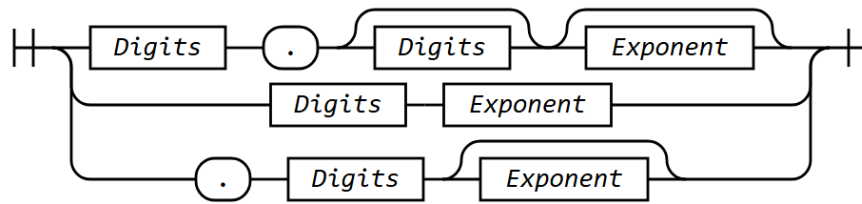
Приклади: `0`, `7`, `42`, `9223372036854775807`.

2.7.2 Дійсні літерали

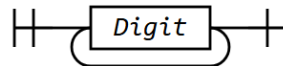
Дійсний літерал має дробову частину, експоненту або обидві. Цілу або дробову частину (але не обидві) можна пропустити. Дійсний літерал має тип `float`; значення, що після округлення до найближчого значення типу `float` дає нескінченність, є помилкою трансляції CE-10.

```
FloatLit    = Digits '.' [ Digits ] [ Exponent ]
              | Digits Exponent
              | '.' Digits [ Exponent ]
Digits      = Digit { Digit }
Exponent    = ( 'e' | 'E' ) [ '+' | '-' ] Digits
```

FloatLit



Digits



Exponent

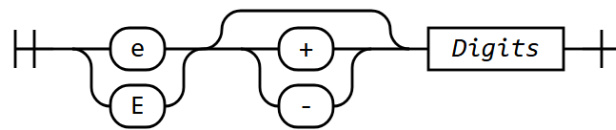


Рисунок 2.8 – Синтаксичні діаграми дійсного літерала

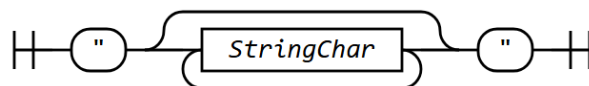
Приклади: 3.14, 2., .5, 1e3, 6.02E+23, 1.5e-3, 007.5.

2.7.3 Рядкові літерали

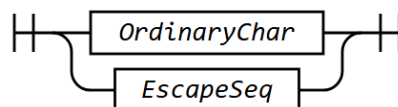
Рядковий літерал — послідовність символів у подвійних лапках. Він не може містити символ кінця рядка. Порожній рядок записується як `""`. Для запису спеціальних символів використовуються escape-послідовності (табл. 2.1); будь-яка інша послідовність, що починається зі зворотної косої риски, є лексичною помилкою CE-01.

```
StringLit    = '"' { StringChar } '"'
StringChar   = OrdinaryChar | EscapeSeq
EscapeSeq    = '\\' ( 'n' | 't' | 'r' | '"' | '\\' )
OrdinaryChar = ? будь-який символ Unicode, крім '"', '\\', '\n' та '\r' ?
```

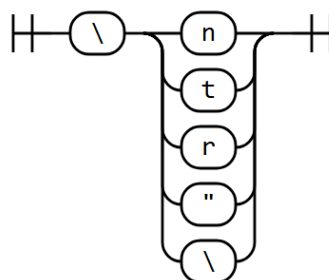
StringLit



StringChar



EscapeSeq



OrdinaryChar

— ? будь-який символ Unicode, крім "'", '\', '\n' та '\r' ? —

Рисунок 2.9 – Синтаксичні діаграми рядкового літерала

Таблиця 2.1 – Escape-послідовності рядкових літералів

Послідовність	Символ	Код Unicode
\n	новий рядок	U+000A
\t	горизонтальна табуляція	U+0009
\r	повернення каретки	U+000D
\"	подвійні лапки	U+0022
\\	зворотна коса риска	U+005C

Приклади: "", "Krok", "Привіт, світе!", "стовпчик 1\tстовпчик 2\n",
"лапки: \"x\"", коса риска: \\".

2.7.4 Логічні літерали

Логічні літерали true та false мають тип bool.

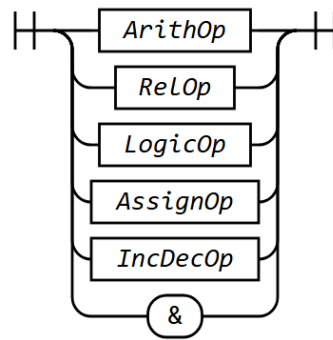
BoolLit = true | false

2.8 Оператори та роздільники

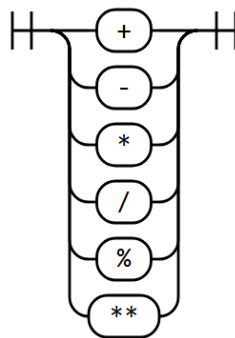
Нижче наведено їх повний перелік; семантику операторів описано в розд. 4.4.

Operator	=	ArithOp		RelOp		LogicOp		AssignOp		IncDecOp		'&'
ArithOp	=	'+' '-' '*' '/' '%' '**'										
LogicOp	=	'&&' ' ' '!'										
RelOp	=	'==' '!=' '<' '<=' '>' '>='										
AddOp	=	'+' '-'										
MulOp	=	'*' '/' '%'										
UnaryOp	=	'-' '+' '!'										
AssignOp	=	'=' ':='										
IncDecOp	=	'++' '--'										
Delimiter	=	'(' ')' '[' ']' '{' '}' ',' ';' '.'										

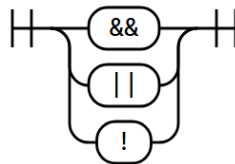
Operator



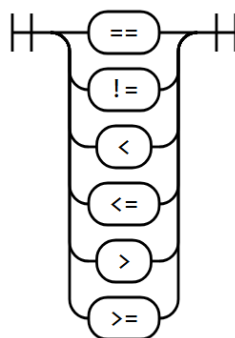
ArithOp



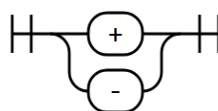
LogicOp



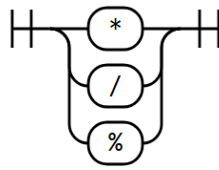
RelOp



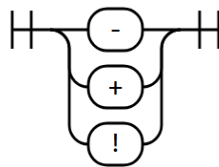
AddOp



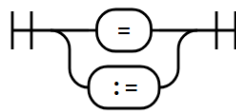
MulOp



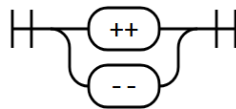
UnaryOp



AssignOp



IncDecOp



Delimiter

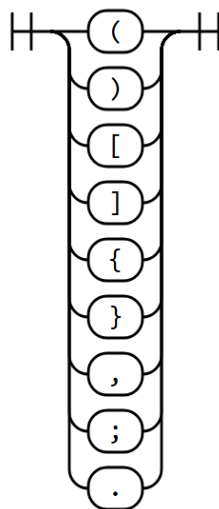


Рисунок 2.11 – Синтаксичні діаграми операторів та роздільників

Лексеми `+`, `-` та `*` мають кілька ролей, які розрізняє синтаксичний аналізатор за контекстом: `+` і `-` — бінарні або унарні оператори, `*` — множення, розіменування (`*p`) або позначка типу параметра-вказівника (`*int`). Правила

AddOp, MulOp та UnaryOp виділяють ті групи операторів, які використовуються у відповідних правилах граматики виразів.

2.9 Автоматична вставка крапки з комою

Грамматика мови використовує токен `;` як завершувач оголошень та інструкцій. Як і в мові Go, у тексті програми крапку з комою зазвичай не пишуть: її вставляє лексичний аналізатор за таким правилом.

Коли лексичний аналізатор зустрічає роздільник рядків або кінець файлу, він вставляє в потік токенів `;`, якщо останній виділений у цьому рядку токен є одним із таких:

- ідентифікатор;
- цілий, дійсний, рядковий або логічний літерал;
- ключове слово `return`, `int`, `float`, `bool` або `string`;
- оператор `++` чи `--`;
- закриваюча дужка `)`, `]` або `}`.

Коментар до кінця рядка на правило не впливає. Явно записана крапка з комою дозволяє розмістити кілька інструкцій в одному рядку й обов'язкова в заголовку трикомпонентної інструкції `for`.

Наслідки правила:

- фігурна дужка, що відкриває блок, повинна стояти в тому самому рядку, що й `if`, `for`, `func` чи `struct`;
- ключове слово `else` повинно стояти в тому самому рядку, що й закриваюча дужка `}` попереднього блоку, інакше після `}` буде вставлено `;` і виникне синтаксична помилка;
- список елементів літерала масиву, розбитий на кілька рядків, має закінчуватися комою перед переходом на новий рядок.

```
if x > 0 {           // правильно
    y = 1
} else {            // правильно: else у рядку з }
    y = 2
}
for i := 0; i < 3; i++ { fmt.Print(i); fmt.Print(" ") } // явні ;
```

2.10 Таблиця токенів

Перелік усіх лексем мови Krok наведено в табл. 2.2. Таблицю побудовано за граматиною мови і вона містить усі термінали, що зустрічаються в її правилах; кожен запис відповідає рівно одній лексемі. Один приклад лексеми в рядку означає, що лексема унікальна; текст «символ ...» означає, що лексема є термінальним символом граматики. Саме ця таблиця є вхідними даними для побудови лексичного аналізатора.

Колонка «Клас» подає класифікацію лексем, прийняту в [2]: **TRM** — ключові слова, оператори та роздільники (термінальні лексеми, для яких достатньо коду); **IDN** — ідентифікатори; **CON** — константи (літерали). Лексеми класів IDN і CON, крім коду, потребують індексу у відповідній таблиці ідентифікаторів або констант, яку буде лексичний аналізатор. Останні чотири рядки таблиці (коментар, пробільні символи, роздільник рядків) позначено знаком «—»: вони не є токенами і синтаксичному аналізатору не передаються, але наведені для повноти опису лексичного рівня, оскільки роздільник рядків керує вставкою ; (розд. 2.9).

Таблиця 2.2 – Таблиця лексем мови Krok

Код	Приклади лексем	Токен	Клас	Неформальний опис
1	x, total, Point, tmp1	id	IDN	ідентифікатор
2	0, 42, 1000	intnum	CON	ціле без знака
3	3.14, 2., .5, 1e3, 6.02E+23	floatnum	CON	дійсне без знака
4	"" , "Krok", "a\tb\n"	strlit	CON	рядковий літерал
5	true	boolval	CON	логічне значення
6	false	boolval	CON	логічне значення
7	package	keyword	TRM	символ package
8	import	keyword	TRM	символ import
9	func	keyword	TRM	символ func
10	var	keyword	TRM	символ var
11	const	keyword	TRM	символ const
12	type	keyword	TRM	символ type
13	struct	keyword	TRM	символ struct
14	if	keyword	TRM	символ if
15	else	keyword	TRM	символ else
16	for	keyword	TRM	символ for
17	return	keyword	TRM	символ return
18	int	keyword	TRM	символ int
19	float	keyword	TRM	символ float

Код	Приклади лексем	Токен	Клас	Неформальний опис
20	bool	keyword	TRM	символ bool
21	string	keyword	TRM	символ string
22	len	keyword	TRM	символ len
23	fmt	keyword	TRM	символ fmt
24	Print	keyword	TRM	символ Print
25	Println	keyword	TRM	символ Println
26	Scan	keyword	TRM	символ Scan
27	+	add_op	TRM	додавання, унарний плюс, конкатенація
28	-	add_op	TRM	віднімання, унарний мінус
29	*	mult_op	TRM	множення; розмінування
30	/	mult_op	TRM	ділення
31	%	mult_op	TRM	остача від ділення
32	**	pow_op	TRM	піднесення до степеня
33	==	rel_op	TRM	дорівнює
34	!=	rel_op	TRM	не дорівнює
35	<	rel_op	TRM	менше
36	<=	rel_op	TRM	менше або дорівнює
37	>	rel_op	TRM	більше
38	>=	rel_op	TRM	більше або дорівнює
39	&&	and_op	TRM	логічне «і»
40		or_op	TRM	логічне «або»
41	!	not_op	TRM	логічне заперечення
42	=	assign_op	TRM	присвоєння
43	:=	define_op	TRM	оголошення з ініціалізацією
44	++	incdec_op	TRM	інкремент
45	--	incdec_op	TRM	декремент
46	&	addr_op	TRM	взяття адреси
47	(	paren	TRM	ліва кругла дужка
48	)	paren	TRM	права кругла дужка
49	[	bracket	TRM	ліва квадратна дужка
50	]	bracket	TRM	права квадратна дужка
51	{	brace	TRM	ліва фігурна дужка
52	}	brace	TRM	права фігурна дужка
53	,	punct	TRM	кома
54	;	punct	TRM	крапка з комою (явна або вставлена)
55	.	punct	TRM	крапка (доступ до поля, fmt.)
56	// коментар	comment	—	однорядковий коментар (ігнорується)
57	\32	ws	—	пробіл (ігнорується)
58	\t	ws	—	горизонтальна табуляція (ігнорується)
59	\n, \r\n	eol	—	кінець рядка (ігнорується; може породити ;)

3 Типи даних і змінні

3.1 Значення, змінні та категорії змінних

Значення мовою Krok подаються або літералами (розд. 2.7), або значеннями змінних і констант, або результатами обчислення виразів. Кожне значення, змінна та вираз мають тип, відомий на етапі трансляції (статична типізація).

Змінна — іменована ділянка пам'яті, значення якої може змінюватися під час виконання інструкціями присвоєння, інкременту/декременту та введення. За місцем оголошення змінні поділяються на категорії, наведені в табл. 3.1.

Таблиця 3.1 – Категорії змінних мови Krok

Категорія	Де оголошується	Розміщення	Час життя	Початкове значення
глобальна змінна	блок пакета, <code>var</code>	статична область	усе виконання програми	нульове або значення константного ініціалізатора
локальна змінна	тіло функції або вкладений блок, <code>var</code> чи <code>:=</code>	фрейм стека	від оголошення до виходу з блоку	нульове або значення ініціалізатора
параметр-значення	список параметрів функції, тип <code>T</code>	фрейм стека	виконання виклику	копія значення аргументу
параметр-вказівник	список параметрів функції, тип <code>*T</code>	фрейм стека	виконання виклику	адреса змінної викликача
іменована константа	блок пакета або тіло функції, <code>const</code>	пам'яті не займає	—	значення константного виразу

В інструкції присвоєння змінна використовується по-різному залежно від того, стоїть вона ліворуч чи праворуч від оператора `=`. Значення ліворуч називають l-значенням (*lvalue*) — це вказівник на ділянку пам'яті; значення праворуч називають r-значенням (*rvalue*) — це власне значення (розд. 4.5).

Обов'язковість оголошення. Кожен ідентифікатор, крім імені пакета `main`, має бути оголошений (розд. 6); звернення до неоголошеного імені є помилкою SE-03. Змінна, оголошена без ініціалізатора, отримує нульове

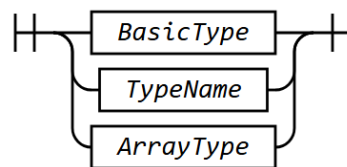
значення свого типу, тому читання змінної до першого присвоєння не є помилкою.

3.2 Класифікація типів

Типи мови Krok поділяються на базові (примітивні), користувацькі скалярні та сконструйовані. Стандартних (бібліотечних) типів, які постачалися б разом із мовою, Krok не має: перелік доступних типів вичерпується наведеними нижче.

```
Type      = BaseType | TypeName | ArrayType
BasicType  = int | float | bool | string
```

Type



BasicType

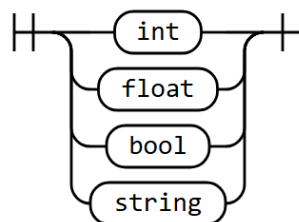


Рисунок 3.1 – Синтаксичні діаграми типів

Базовими є чотири скалярні типи, наведені в табл. 3.2.

Таблиця 3.2 – Базові (примітивні) типи

Тип	Множина значень	Нульове значення	Літерали
int	цілі числа від −9223372036854775808 до 9223372036854775807 (64-бітне доповняльне представлення)	0	IntLit
float	числа з рухомою комою подвійної точності IEEE 754 (binary64), крім нескінченностей і NaN	0.0	FloatLit
bool	false, true	false	true, false

Тип	Множина значень	Нульове значення	Літерали
string	незмінні послідовності символів Unicode довільної довжини	""	StringLit

Мінімальне значення типу `int` не можна записати одним літералом; його отримують виразом `-9223372036854775807 - 1`.

3.3 Користувацькі скалярні типи

Оголошення `type T U` (розд. 6.3), де `U` — базовий скалярний тип або інший користувацький скалярний тип, створює новий **іменований тип** `T` з тією самою множиною значень і операцій, що й у **базового типу** `U` (для ланцюжка оголошень базовим є вбудований тип у кінці ланцюжка).

```
TypeName = Ident
```

TypeName

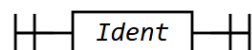


Рисунок 3.2 – Синтаксична діаграма імені типу

Іменований тип відмінний від свого базового типу та від будь-якого іншого типу (номінальна еквівалентність): змінній типу `Celsius` не можна присвоїти значення типу `float` без явного приведення `Celsius(x)`. Виняток — літерали: числовий, рядковий або логічний літерал (зокрема зі знаком `+` чи `-`) неявно набуває іменованого типу, якщо його тип збігається з базовим типом або може бути до нього розширений (розд. 3.6.1).

```

type Celsius float
type Fahrenheit float
var t Celsius = 36.6           // літерал набуває типу Celsius
var f Fahrenheit = Fahrenheit(t*9/5 + 32)
// t = f                       // CE-05: різні іменовані типи

```

Бінарна арифметична операція над двома операндами одного іменованого типу дає результат того самого типу.

3.4 Сконструйовані типи

Сконструйованими є типи масивів, структурні типи та типи параметрів-вказівників.

3.4.1 Масиви

Тип масиву $[N]T$ описує послідовність рівно N елементів типу T , де N — константний вираз типу `int` зі значенням більше нуля (розд. 4.6), а T — будь-який тип. Багатовимірний масив — масив масивів: `[3][4]int` містить 3 елементи типу `[4]int`.

```
ArrayType = '[' Expression ']' Type
```

ArrayType

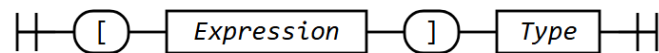


Рисунок 3.3 – Синтаксична діаграма типу масиву

- **Ініціалізація.** Змінна-масив без ініціалізатора отримує масив нульових значень елементів. Статична ініціалізація виконується літералом масиву $[N]T\{e_1, \dots, e_k\}$, де $k \leq N$; елементи з індексами від k до $N-1$ отримують нульові значення. Для масиву масивів внутрішні списки записуються без типу: `[2][2]int{{1, 2}, {3, 4}}`. Кількість елементів $k > N$ є помилкою SE-11. Динамічна ініціалізація виконується присвоєнням значень елементам під час виконання (наприклад, у циклі).
- **Індексування.** Вираз `a[i]` позначає елемент масиву `a` з індексом `i`, де `i` — вираз типу `int`. Елементи нумеруються від 0 до $N-1$. Якщо значення індексу виходить за ці межі, виникає помилка часу виконання RE-04. Якщо індекс є константним виразом, вихід за межі виявляється на етапі трансляції (SE-11).
- **Довжина.** Вираз `len(a)` має тип `int` і дорівнює N ; для масиву він є константним виразом.
- **Семантика значень.** Масиви є типами-значеннями: присвоєння, ініціалізація, передача аргументу та повернення результату копіюють усі елементи.
- Масиви не порівнюються операторами `==`, `!=` і не виводяться операцією `fmt.Print` безпосередньо (SE-05).

3.4.2 Структури

Оголошення `type T struct { ... }` створює структурний тип `T` із набором іменованих полів.

```
StructType    = struct '{' FieldList '}'  
FieldList     = [ FieldDecl FieldTail ]  
FieldTail     = [ ';' FieldList ]  
FieldDecl     = IdentList Type
```

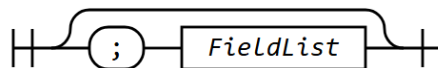
StructType



FieldList



FieldTail



FieldDecl

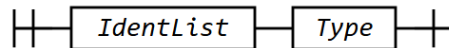


Рисунок 3.4 – Синтаксичні діаграми структурного типу

- Імена полів однієї структури мають бути унікальними (CE-12). Тип поля може бути будь-яким типом, крім самої структури `T` безпосередньо або через масиви чи інші структури, оскільки розмір такого значення був би нескінченним (CE-12).
- **Доступ до полів.** Вираз `s.f` позначає поле `f` значення `s` структурного типу. Звернення до неіснуючого поля є помилкою CE-12. Доступ до поля вкладеної структури записується ланцюжком: `seg.from.x`.
- **Ініціалізація.** Змінна структурного типу отримує значення, в якому кожне поле має нульове значення свого типу. Літералів структур мови не має; поля заповнюються присвоєннями.
- **Семантика значень.** Присвоєння структури копіює всі поля (зокрема вкладені масиви і структури) поелементно.

- Структури не порівнюються операторами `==`, `!=` і не виводяться безпосередньо (CE-05).

3.4.3 Параметри-вказівники

Тип `*T` («вказівник на `T`») може бути лише типом параметра функції. Він реалізує передачу параметра за **посиланням**.

`ParamType` = ['*'] `Type`

ParamType

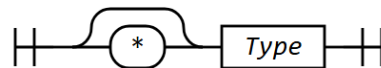


Рисунок 3.5 – Синтаксична діаграма типу параметра

- Аргументом для параметра типу `*T` є вираз `&x`, де `x` — адресований вираз (розд. 4.5) типу, ідентичного `T`, або інший параметр типу `*T`. Неявне розширення `int` $\rightarrow$ `float` для вказівників не застосовується.
- Вираз `*p`, де `p` — параметр типу `*T`, є l-значенням типу `T`, що позначає ту саму змінну, адресу якої передано у виклику. Суфікси застосовуються до розіменованого значення: `*p[i]` означає `(*p)[i]`, `*p.x` означає `(*p).x`.
- Параметру-вказівнику не можна присвоїти нове значення; вказівники не порівнюються, не виводяться і не беруть участі в арифметиці.
- Значення «порожнього вказівника» (`nil`) у мові немає: кожен параметр-вказівник завжди посилається на існуючу змінну. Оскільки вказівник не можна зберегти в змінній, полі чи результаті функції, він не може пережити змінну, на яку посилається.

3.5 Еквівалентність та сумісність типів

Два типи **ідентичні**, якщо це той самий базовий тип, той самий іменований (користувацький скалярний, іменований масивний або структурний) тип, або якщо обидва є неіменованими типами масивів з однаковою довжиною та ідентичними типами елементів. Отже, для іменованих

типів діє номінальна еквівалентність, а для неіменованих типів масивів — структурна.

Значення x типу V **сумісне за присвоєнням** із типом T (може бути присвоєне змінній типу T , передане параметру типу T , повернене як результат типу T або використане як елемент масиву з елементами типу T), якщо виконується одна з умов:

1. V і T ідентичні;
2. V — `int`, T — `float` (неявне розширення, розд. 3.6.1);
3. x — літерал (можливо, з унарним `+` чи `-`), а T — іменований скалярний тип, до базового типу якого літерал сумісний за умовами 1–2;
4. V — неіменований тип масиву, T — іменований тип масиву з ідентичною структурою.

3.6 Приведення типів

3.6.1 Неявне приведення

Єдине неявне приведення в мові — **розширення** значення типу `int` до типу `float`: ціле число інтерпретується як дійсне з нульовою дробовою частиною, округлене до найближчого значення `float`, якщо воно не представляється точно. Розширення виконується:

- в арифметичній операції та операції порівняння, якщо один операнд має тип `int`, а інший — `float` (перетворюється операнд типу `int`);
- у присвоєнні та ініціалізації змінної типу `float` значенням типу `int`;
- під час передачі аргументу типу `int` параметру типу `float` та повернення значення типу `int` з функції з результатом типу `float`;
- для елемента типу `int` у літералі масиву з елементами типу `float`.

Звуження `float` → `int` неявно не виконується ніколи: `var n int = 2.5` є помилкою CE-05. Неявні перетворення між `bool`, `string` і числовими типами відсутні.

3.6.2 Явне приведення

Явне приведення записується як $T(x)$, де T — базовий або користувацький скалярний тип (для користувацького типу — синтаксично виклик, який семантичний аналізатор розпізнає за тим, що ідентифікатор позначає тип).

Таблиця 3.3 – Допустимі явні приведення

Тип x	Тип T	Результат
числовий	float	значення x як дійсне число
float	int	значення x з відкинутою дробовою частиною (округлення в бік нуля): $\text{int}(2.9) = 2$, $\text{int}(-2.9) = -2$; якщо результат не належить множині значень <code>int</code> — RE-09
int	int	значення без змін
тип з базовим U	тип з базовим U'	приведення $U \rightarrow U'$ за рядками вище, якщо обидва базові типи числові або однакові

Приведення між `bool`, `string` та числовими типами, а також приведення масивів і структур недопустимі (CE-06).

4 Вирази

4.1 Класифікація виразів

Вираз — послідовність операторів і операндів, що визначає обчислення значення. За типом обчисленого значення вирази поділяються на арифметичні (тип `int` або `float`), логічні (тип `bool`), рядкові (тип `string`) та вирази сконструйованих типів (масиви і структури — результати індексування, доступу до поля чи виклику функції). Окремо виділяють:

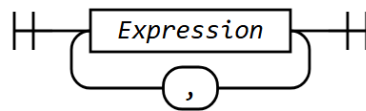
- **первинні вирази** — літерали, імена, розіменування, приведення, виклики, літерали масивів і вирази в дужках;
- **константні вирази** (розд. 4.6) — обчислювані на етапі трансляції;
- **адресовані вирази (І-значення)** (розд. 4.5) — позначають ділянку пам'яті.

4.2 Синтаксис виразів

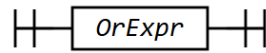
Пріоритет операторів задано стратифікацією граматики: оператори з вищим пріоритетом описано на глибших рівнях вкладеності (`OrExpr` → `AndExpr` → `RelExpr` → `AddExpr` → `MulExpr` → `UnaryExpr` → `PowerExpr` → `PrimaryExpr`). Лівоасоціативні оператори описано повтореннями { ... }, правоасоціативний оператор `**` — правою рекурсією через `UnaryExpr`. Оператори порівняння неасоціативні: запис `a < b < c` є синтаксичною помилкою.

```
ExpressionList = Expression { ',' Expression }
Expression     = OrExpr
OrExpr         = AndExpr { '||' AndExpr }
AndExpr        = RelExpr { '&&' RelExpr }
RelExpr        = AddExpr [ RelOp AddExpr ]
AddExpr        = MulExpr { AddOp MulExpr }
MulExpr        = UnaryExpr { MulOp UnaryExpr }
UnaryExpr      = UnaryOp UnaryExpr | PowerExpr
PowerExpr      = PrimaryExpr [ '**' UnaryExpr ]
```

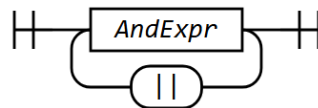
ExpressionList



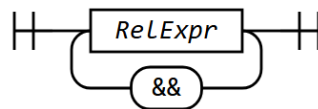
Expression



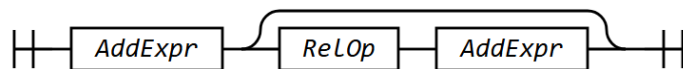
OrExpr



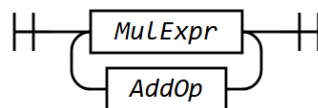
AndExpr



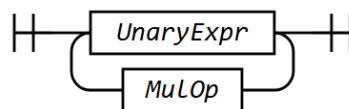
RelExpr



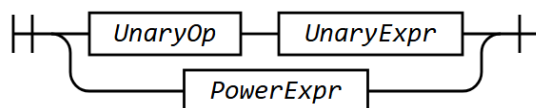
AddExpr



MulExpr



UnaryExpr



PowerExpr

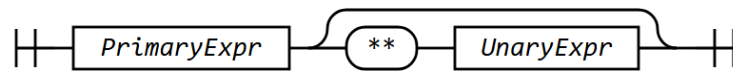
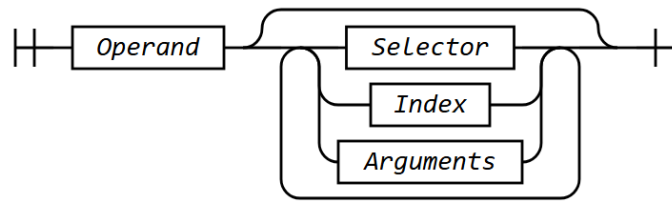


Рисунок 4.1 – Синтаксичні діаграми виразів

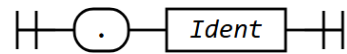
Первинний вираз складається з операнда та нуля або більше суфіксів: доступу до поля (`.x`), індексування (`[i]`) та виклику (`((args))`). Операндом може бути літерал, ідентифікатор, розіменування параметра-вказівника, явне приведення до базового типу, виклик `len`, літерал масиву або вираз у дужках.

```
PrimaryExpr    = Operand { Selector | Index | Arguments }
Selector       = '.' Ident
Index          = '[' Expression ']'
Arguments      = '(' [ ArgumentList ] ')'
ArgumentList   = Argument { ',' Argument }
Argument       = AddrExpr | Expression
AddrExpr       = '&' PrimaryExpr
Operand        = Literal | Ident | Dereference | Conversion | LenCall
               | ArrayLit | '(' Expression ')'
Dereference    = '*' Ident
Conversion     = BasicType '(' Expression ')'
LenCall        = len '(' Expression ')'
```

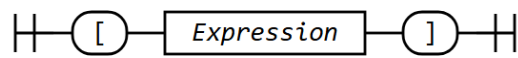
PrimaryExpr



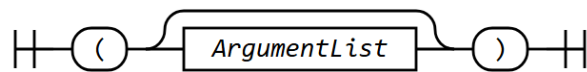
Selector



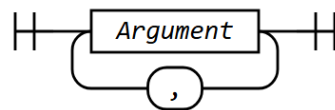
Index



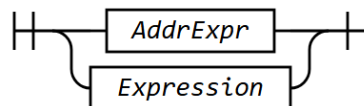
Arguments



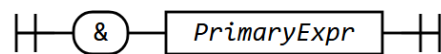
ArgumentList



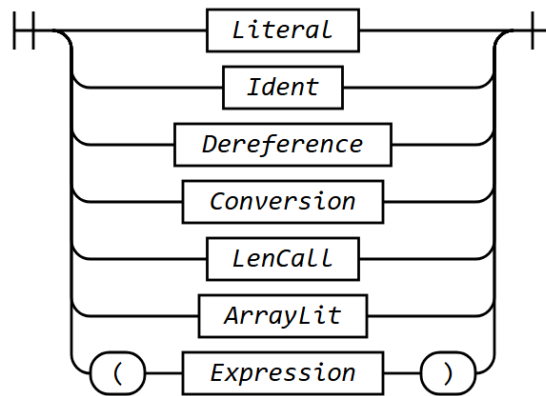
Argument



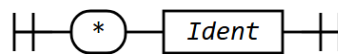
AddrExpr



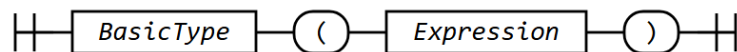
Operand



Dereference



Conversion



LenCall

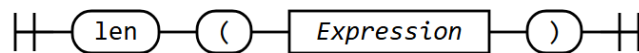
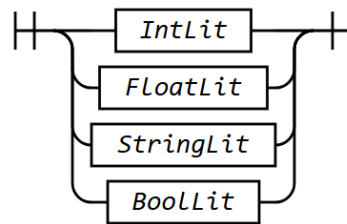


Рисунок 4.2 – Синтаксичні діаграми первинних виразів

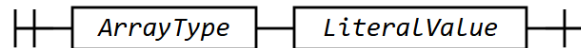
Літерал масиву складається з типу масиву та списку значень елементів у фігурних дужках. Для вкладених масивів тип внутрішніх списків не записується. Після останнього елемента допускається кома.

<i>Literal</i>	= <i>IntLit</i> <i>FloatLit</i> <i>StringLit</i> <i>BoolLit</i>
<i>ArrayLit</i>	= <i>ArrayType</i> <i>LiteralValue</i>
<i>LiteralValue</i>	= '{' [<i>Element</i> <i>ElementTail</i>] '}'
<i>ElementTail</i>	= [',' [<i>Element</i> <i>ElementTail</i>]]
<i>Element</i>	= <i>Expression</i> <i>LiteralValue</i>

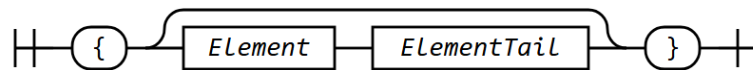
Literal



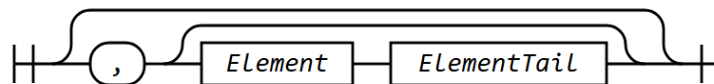
ArrayLit



LiteralValue



ElementTail



Element

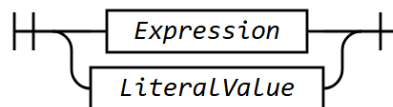


Рисунок 4.3 – Синтаксичні діаграми літералів у виразах

Приклади виразів:

```
-b + 4.5 * c           // (-b) + (4.5 * c)
2 ** 3 ** 2           // 2 ** (3 ** 2) = 512
-2 ** 2               // -(2 ** 2) = -4
x >= 0 && !done || n == 0
float(total) / float(len(v))
[3]int{1, 2, 3}
m[i][j] + p.x * (*ptr)
```

4.3 Обчислення виразів

Операнди бінарних операторів, аргументи викликів, індекси та елементи літералів масивів обчислюються **зліва направо**. Оператори **&&** та **||** обчислюються за скороченою схемою: правий операнд **&&** обчислюється лише

тоді, коли лівий дорівнює true, а правий операнд `||` — лише тоді, коли лівий дорівнює false. Решта операторів обчислює обидва операнди.

Порядок обчислення не залежить від пріоритету операторів: пріоритет визначає лише структуру виразу (дерево розбору), а не послідовність обчислення операндів.

4.4 Оператори: пріоритет, асоціативність, типи операндів і результату

Таблиця 4.1 – Пріоритет та асоціативність операторів

Пріоритет	Оператори	Опис	Асоціативність
8 (найвищий)	<code>.f</code> , <code>[i]</code> , <code>(args)</code> , <code>*p</code>	доступ до поля, індексування, виклик, розіменування	ліва
7	<code>**</code>	піднесення до степеня	права
6	унарні <code>-</code> , <code>+</code> , <code>!</code>	зміна знака, тотожність, заперечення	права (префіксні)
5	<code>*</code> , <code>/</code> , <code>%</code>	мультиплікативні	ліва
4	<code>+</code> , <code>-</code>	адитивні, конкатенація рядків	ліва
3	<code>==</code> , <code>!=</code> , <code><</code> , <code><=</code> , <code>></code> , <code>>=</code>	порівняння	неасоціативні
2	<code>&&</code>	кон'юнкція	ліва
1 (найнижчий)	<code> </code>	диз'юнкція	ліва

Оператор `**` має вищий пріоритет, ніж унарні оператори зліва від нього, але його правий операнд може починатися з унарного оператора: `-2 ** 2` дорівнює `-(2 ** 2) = -4`, а `2.0 ** -1` дорівнює `0.5`. Оператор `&` (взяття адреси) не є оператором виразу: він може стояти лише перед аргументом виклику.

4.4.1 Арифметичні оператори

Таблиця 4.2 – Типи операндів і результатів арифметичних операторів

Оператор	Операція	Типи операндів	Тип результату
<code>+</code>	додавання	<code>int</code> , <code>int</code>	<code>int</code>
<code>-</code> , <code>*</code>	віднімання, множення	<code>int</code> , <code>int</code>	<code>int</code>
<code>+</code> , <code>-</code> , <code>*</code>	те саме	<code>float</code> і <code>float</code> або <code>int</code>	<code>float</code>

Оператор	Операція	Типи операндів	Тип результату
/	ділення	int, int	int (частка, округлена в бік нуля)
/	ділення	float i float або int	float
%	остача	int, int	int
**	ступінь	int, int	int
**	ступінь	float i float або int	float
+	конкатенація	string, string	string
унарні +, -	тотожність, зміна знака	int або float	тип операнда

Для іменованих типів діють ті самі правила відносно базового типу за умови, що обидва операнди мають той самий іменований тип (або один із них — літерал).

- Для цілих a / b і $a \% b$ виконується тотожність $a = (a / b) * b + a \% b$; знак остачі збігається зі знаком a : $-7 / 2 = -3$, $-7 \% 2 = -1$.
- Цілочислове піднесення до степеня $a ** b$ вимагає $b \geq 0$ (інакше RE-05); $0 ** 0 = 1$. Дійсне піднесення до степеня виконується за правилами функції pow стандарту IEEE 754.
- Ділення на нуль (цілочислове чи дійсне) та взяття остачі від ділення на нуль спричиняють RE-01.
- Якщо результат цілочислової операції (зокрема унарного мінуса, ++, - та **) не належить множині значень int, виникає RE-02. Якщо результат дійсної операції є нескінченністю або NaN, виникає RE-03; втрата точності та антипереповнення (округлення до нуля) помилками не є.
- Оператори % для float, а також будь-які арифметичні оператори для bool, масивів і структур недопустимі (CE-05).

4.4.2 Оператори порівняння

Результат порівняння завжди має тип bool.

- Числові операнди (int, float, у разі змішаних типів — з розширенням до float) порівнюються за значенням усіма шістьма операторами.
- Рядки порівнюються всіма шістьма операторами лексикографічно за кодами символів Unicode.

- Логічні значення порівнюються лише операторами `==` та `!=`.
- Операнди іменованих типів порівнюються, якщо мають той самий тип (або один із них — літерал), за правилами базового типу.
- Масиви, структури та вказівники не порівнюються (CE-05).

4.4.3 Логічні оператори

Оператори `&&`, `||` та `!` застосовуються лише до операндів типу `bool` (або іменованого типу з базовим `bool`) і дають результат того самого типу. Семантику скороченого обчислення наведено в розд. 4.3. Числа та рядки не інтерпретуються як логічні значення: `if n { }` для `n` типу `int` є помилкою CE-05.

4.4.4 Операції над рядками

Для рядків визначено конкатенацію `+`, порівняння (розд. 4.4.2) та функцію `len(s)`, яка повертає кількість символів Unicode в рядку (значення типу `int`). Рядки незмінні: індексування рядків не підтримується.

4.5 L-значення

L-значення (адресований вираз) позначає місце зберігання значення і може стояти ліворуч від `=`, бути операндом `++/--`, операндом `&` чи аргументом `fmt.Scan`. L-значеннями є:

- ім'я змінної або параметра, що не є вказівником;
- розіменування параметра-вказівника `*p`;
- індексування `a[i]`, якщо `a` є l-значенням;
- доступ до поля `s.f`, якщо `s` є l-значенням.

Не є l-значеннями: константи, літерали, результати викликів функцій і приведень, вирази з операторами, імена функцій і типів, параметри-вказівники без розіменування.

4.6 Константні вирази

Константний вираз обчислюється на етапі трансляції. Він може містити лише літерали, імена констант, оператори (крім викликів функцій та &), явні приведення базових типів і вирази `len(a)` для масивів. Помилки обчислення константного виразу (ділення на нуль, переповнення, від'ємний цілий показник степеня) є помилками трансляції CE-10.

Константні вирази обов'язкові у трьох контекстах: довжина масиву, ініціалізатор іменованої константи та ініціалізатор глобальної змінної.

5 Інструкції

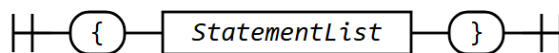
Інструкції визначають алгоритмічні дії, які мають бути виконані у програмі.

5.1 Блок і список інструкцій

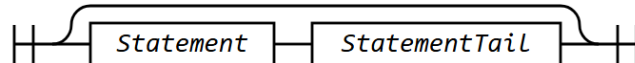
Послідовність інструкцій записується в блоці у фігурних дужках; інструкції розділяються токеном `;` (явним або вставленим за правилом розд. 2.9). Оголошення змінних і констант усередині блоку також є інструкціями.

```
Block      = '{' StatementList '}'  
StatementList = [ Statement StatementTail ]  
StatementTail = [ ';' StatementList ]  
Statement   = VarDecl | ConstDecl | IfStmt | ForStmt | ReturnStmt  
            | IoStmt | Block | SimpleStmt
```

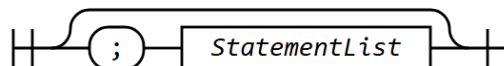
Block



StatementList



StatementTail



Statement

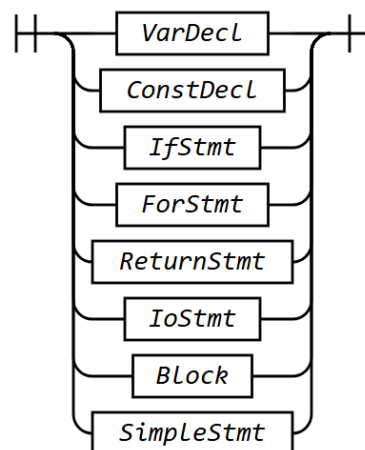


Рисунок 5.1 – Синтаксичні діаграми блоку та інструкцій

Інструкції блоку виконуються послідовно в порядку запису. Блок утворює нову область видимості (розд. 6.5).

5.2 Присвоювання значення

`SimpleStmt` = `Expression` [`AssignOp` `Expression` | `IncDecOp`]

SimpleStmt



Рисунок 5.2 – Синтаксична діаграма простої інструкції

Проста інструкція `SimpleStmt` починається з виразу, після якого може стояти оператор присвоєння або оголошення з ініціалізацією та другий вираз, або оператор інкременту чи декременту. Граматика допускає ширшу множину ланцюжків, ніж мова: вимоги до лівої частини присвоєння та до інструкції-виразу є контекстними умовами. Це дозволяє розрізняти присвоєння, оголошення і виклик функції з одним токеном попереднього перегляду.

Інструкція `v = e` обчислює вирази-індекси в лівій частині (зліва направо), потім вираз `e`, після чого записує значення `e` у місце, позначене `l`-значенням `v`. Ліва частина має бути `l`-значенням (CE-07), а значення `e` — сумісним за присвоєнням з типом `v` (CE-05). Для масивів і структур присвоєння копіює значення повністю.

Інструкції `v++` і `v--` еквівалентні `v = v + 1` і `v = v - 1` з однократним обчисленням `v`. Операнд має бути `l`-значенням числового типу.

Інструкція `x := e` оголошує нову змінну (розд. 6.1). Простою інструкцією, що складається лише з виразу, може бути тільки виклик функції (CE-17); результат функції в такому разі відкидається.

Приклади: `x = 5`, `a[i] = a[i] * 2`, `p.x = 1.0`, `*ptr = 0`, `total := 0`, `i++`, `k--`, `swap(&a, &b)`.

5.3 Інструкція розгалуження

Інструкція `if` виконує блок, якщо умова істинна. Необов'язкова гілка `else` містить або блок, або наступну інструкцію `if`, що утворює ланцюжок `else if`.

```
IfStmt      = if Expression Block [ else ( IfStmt | Block ) ]
```

IfStmt

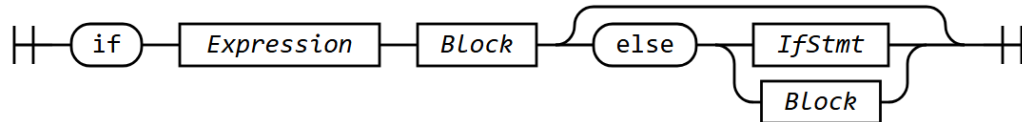


Рисунок 5.3 – Синтаксична діаграма інструкції розгалуження

Фігурні дужки навколо гілок обов'язкові. Завдяки цьому граматики не має проблеми «висячого `else`»: кожне `else` однозначно належить до `if`, блок якого закривається безпосередньо перед ним.

Під час виконання обчислюється умова, яка має тип `bool` (CE-05). Якщо її значення `true`, виконується блок після умови, інакше — гілка `else`, якщо вона є. Ланцюжок `if ... else if ... else` перевіряє умови по черзі і виконує рівно одну гілку: першу, умова якої істинна, або гілку `else`.

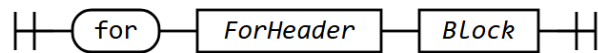
```
if t < 0 {  
    fmt.Println("мороз")  
} else if t < 15 {  
    fmt.Println("прохолодно")  
} else {  
    fmt.Println("тепло")  
}
```

5.4 Інструкція повторення

Інструкція `for` має дві форми. **Цикл із передумовою** містить після `for` лише умову. **Трикомпонентний цикл** (цикл із лічильником) містить ініціалізацію, умову та інструкцію наступного кроку, розділені явними `;`.

```
ForStmt      = for ForHeader Block  
ForHeader    = SimpleStmt [ ';' Expression ';' SimpleStmt ]
```

ForStmt



ForHeader

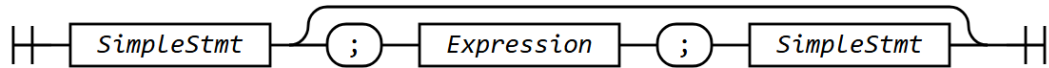


Рисунок 5.4 – Синтаксичні діаграми інструкції повторення

Цикл із передумовою `for E В` виконується так:

1. Обчислюється умова `E` типу `bool` (CE-05).
2. Якщо значення `false`, виконання інструкції завершується.
3. Виконується тіло `В`, після чого керування переходить до кроку 1.

Тіло може не виконатися жодного разу.

Трикомпонентний цикл `for S1; E; S2 В` створює неявний блок і виконується так:

1. Один раз виконується ініціалізаційна інструкція `S1` (присвоєння, оголошення `:=`, `++/--` або виклик). Змінні, оголошені в `S1`, видимі в `E`, `S2` та `В`.
2. Обчислюється умова `E` типу `bool`; якщо значення `false`, виконання інструкції завершується.
3. Виконується тіло `В`.
4. Виконується інструкція наступного кроку `S2` (присвоєння, `++/--` або виклик; оголошення `:=` недопустиме — CE-17), після чого керування переходить до кроку 2.

Змінна циклу — звичайна змінна: її значення можна змінювати в тілі циклу, і ці зміни впливають на подальші ітерації. Кількість ітерацій заздалегідь не обчислюється. Після завершення циклу змінні з `S1` недоступні.

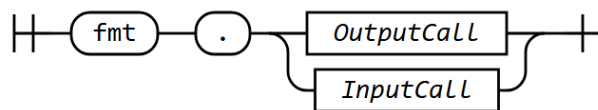
```
for n > 0 {                               // цикл із передумовою
    n = n / 10
}
for k := 10; k > 0; k-- {                 // трикомпонентний цикл
    k = k - 2                             // дозволено: виведе 8 5 2 -1
    fmt.Print(k, " ")
}
```

5.5 Інструкції введення та виведення

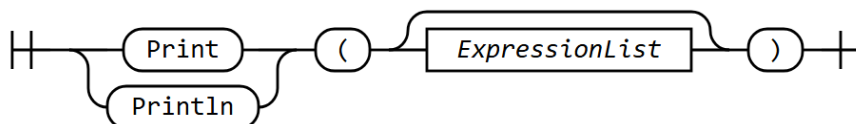
Введення та виведення виконуються операціями вбудованого пакета `fmt`; їх використання без оголошення `import "fmt"` є помилкою CE-14. Аргументами `Scan` можуть бути лише адреси змінних (вирази з оператором `&`), що виражено граматикою.

```
IoStmt      = fmt '.' ( OutputCall | InputCall )
OutputCall  = ( 'Print' | 'Println' ) '(' [ ExpressionList ] ')'
InputCall   = 'Scan' '(' AddrExpr { ',' AddrExpr } ')'
```

IoStmt



OutputCall



InputCall

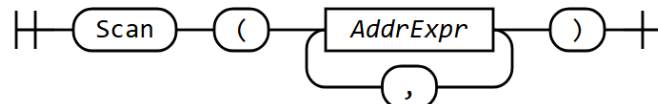


Рисунок 5.5 – Синтаксичні діаграми інструкцій введення та виведення

Виведення. `fmt.Print(e1, ..., en)` записує текстові представлення значень у стандартний потік виведення без роздільників і без переходу на новий рядок. `fmt.Println(e1, ..., en)` записує представлення, розділені одним пробілом, і завершує виведення символом нового рядка; `fmt.Println()` виводить лише символ нового рядка. Аргументи мають бути скалярних типів (CE-05). Текстові представлення значень:

- `int` — десятковий запис, від'ємні числа — зі знаком `-`: 42, -7;
- `float` — найкоротший десятковий запис, який однозначно відновлює значення; якщо $1e-4 \leq |x| < 1e16$, запис має звичайну форму і

завжди містить крапку та принаймні одну цифру після неї (5.0, 0.125, 43.6), інакше — експоненційну форму (1e+16, 2.5e-05);

- `bool` — `true` або `false`;
- `string` — символи рядка без лапок; `escape`-послідовності виводяться як відповідні символи;
- значення іменованого типу — за правилом базового типу.

Введення. `fmt.Scan(&v1, ..., &vn)` послідовно для кожного аргументу пропускає пробільні символи (пробіл, табуляцію, кінець рядка) у стандартному потоці введення, читає слово — найдовшу послідовність непробільних символів — і перетворює його відповідно до типу змінної:

- `int` — необов'язковий знак і десяткові цифри (ведучі нулі допускаються); значення має належати множині значень `int`;
- `float` — необов'язковий знак і запис цілого або дійсного числа за формою `IntLit/FloatLit` (з допуском ведучих нулів);
- `bool` — слово `true` або `false`;
- `string` — слово без змін (рядок, що містить пробіли, одним викликом прочитати не можна);
- іменований тип — за правилом базового типу.

Прочитане значення одразу присвоюється змінній. Змінні масивних і структурних типів як цілі не вводяться (CE-05), але їхні скалярні елементи та поля — вводяться. Якщо слово не можна перетворити на значення потрібного типу, виникає RE-07; якщо потік введення закінчився раніше, ніж прочитано всі значення, — RE-08.

Перед кожною операцією введення та перед завершенням програми буфер виведення примусово записується в потік, тому запрошення до введення з'являються до очікування даних.

Приклади: `fmt.Print("x = ", x)`, `fmt.Println(a, b, a + b)`,
`fmt.Println(), fmt.Scan(&n, &v[i], &p.x)`.

5.6 Інструкція повернення з функції

```
ReturnStmt    = return [ Expression ]
```

ReturnStmt

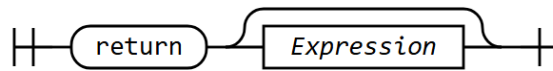


Рисунок 5.6 – Синтаксична діаграма інструкції повернення

Інструкція `return` завершує виконання поточної функції. У функції без результату вона записується без виразу (CE-09); у функції з результатом — з виразом, сумісним за присвоєнням з типом результату, і це значення стає значенням виклику.

Функція з результатом повинна завершуватися інструкцією `return` на кожному шляху виконання (CE-09). Це перевіряється статично: список інструкцій вважається **завершальним**, якщо його остання інструкція є `return`, завершальним блоком або інструкцією `if` з гілкою `else`, у якій усі гілки завершальні. Цикли завершальними не вважаються. Функція без результату повертає керування після виконання останньої інструкції тіла.

Приклади: `return`, `return n * factorial(n-1)`.

6 Декларації

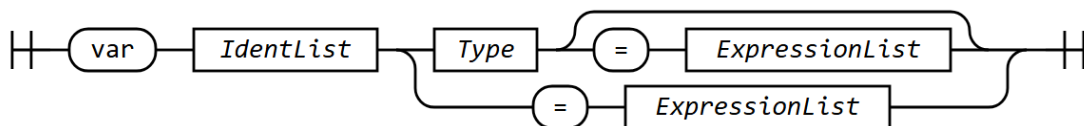
Декларації вводять імена змінних, констант, типів і функцій та визначають їхні властивості. Синтаксично оголошення змінних і констант відрізняються ключовим словом, а також тим, що константа завжди має ініціалізатор і оголошує рівно одне ім'я.

Повторне оголошення імені в тому самому блоці (зокрема імені параметра на верхньому рівні тіла функції) є помилкою CE-04.

6.1 Оголошення змінних

```
VarDecl    = var IdentList  
            ( Type [ '=' ExpressionList ] | '=' ExpressionList )  
IdentList  = Ident { ',', Ident }
```

VarDecl



IdentList

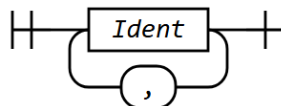


Рисунок 6.1 – Синтаксичні діаграми оголошення змінних

Оголошення `var` має три форми:

- `var a, b T` — змінні типу `T` з нульовими значеннями;
- `var a, b T = e1, e2` — значення виразів мають бути сумісними за присвоєнням з типом `T`;
- `var a, b = e1, e2` — тип кожної змінної збігається з типом відповідного виразу.

Кількість виразів має дорівнювати кількості імен (CE-05). Вирази обчислюються зліва направо до того, як імена стають видимими. Ініціалізатори глобальних змінних мають бути константними виразами (CE-16).

Оголошення з ініціалізацією. Інструкція `x := e` оголошує в поточному блоці нову змінну `x` з типом виразу `e` і присвоює їй значення `e`. Ліва частина має бути ідентифікатором, не оголошеним у поточному блоці (CE-04, CE-17). Вираз `e` не може бути викликом функції без результату (CE-15). Оголошення `:=` дозволене лише всередині функцій.

```
var count int
var x, y float = 1.5, -2
var name = "Krok"
total := 0
```

6.2 Іменовані константи

`ConstDecl` = `const` `Ident` [`Type`] '=' `Expression`

ConstDecl

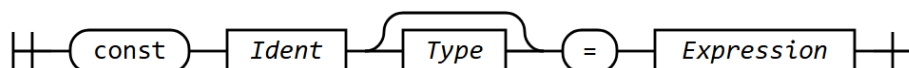


Рисунок 6.2 – Синтаксична діаграма оголошення константи

Оголошення `const C T = e` або `const C = e` вводить ім'я `C` для значення константного виразу `e` (розд. 4.6), обчисленого на етапі трансляції. Тип константи — `T` (значення `e` має бути з ним сумісним) або тип виразу `e`. Константи можуть мати лише скалярні типи.

Константа не є змінною: її не можна змінити присвоєнням, `++/--` чи введенням, і до неї не можна застосувати оператор `&` (CE-07). Оскільки значення константи відоме транслятору, вона може використовуватися там, де потрібен константний вираз, — зокрема як довжина масиву.

Синтаксис оголошення запозичено з мови-прототипу; для порівняння в табл. 6.1 наведено запис іменованих констант у кількох мовах програмування.

Таблиця 6.1 – Іменовані константи в різних мовах програмування

Мова	Запис	Особливість
Pascal	<code>const pi = 3.14159;</code>	тип виводиться з літерала
C++	<code>const double PI = 3.14159;</code>	<code>constexpr</code> — обчислення на етапі компіляції
Java	<code>public static final double PI = 3.14159;</code>	окремого поняття константи немає

Мова	Запис	Особливість
C#	public const double Pi = 3.14159;	readonly — ініціалізація під час виконання
Go	const Pi = 3.14159	тип необов'язковий, крапка з комою не потрібна
Krok	const Pi = 3.14159 або const Size int = 5	тип необов'язковий; лише скалярні типи; лише константні вирази

```
const Size int = 5
const Eps = 1.5e-3
const Greeting = "Привіт!"
var table [Size]int // константа як довжина масиву
```

6.3 Оголошення типів

```
TypeDecl = type Ident ( Type | StructType )
```

TypeDecl

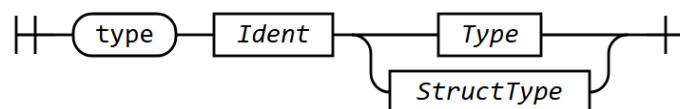


Рисунок 6.3 – Синтаксична діаграма оголошення типу

Оголошення `type` вводить ім'я користувацького скалярного типу (розд. 3.3), іменованого типу масиву або структурного типу (розд. 3.4.2). Оголошення типів дозволені лише на верхньому рівні програми, що виражено граматикою.

```
type Celsius float
type Row [3]int
type Point struct {
    x, y float
    label string
}
```

6.4 Оголошення функцій

Оголошення функції задає її ім'я, список параметрів, необов'язковий тип результату та тіло. Кілька послідовних параметрів одного типу можна згрупувати: `a, b int`.

```
FuncDecl = func Ident '(' [ ParamList ] ')' [ Type ] Block
ParamList = ParamGroup { ',' ParamGroup }
ParamGroup = IdentList ParamType
```

FuncDecl

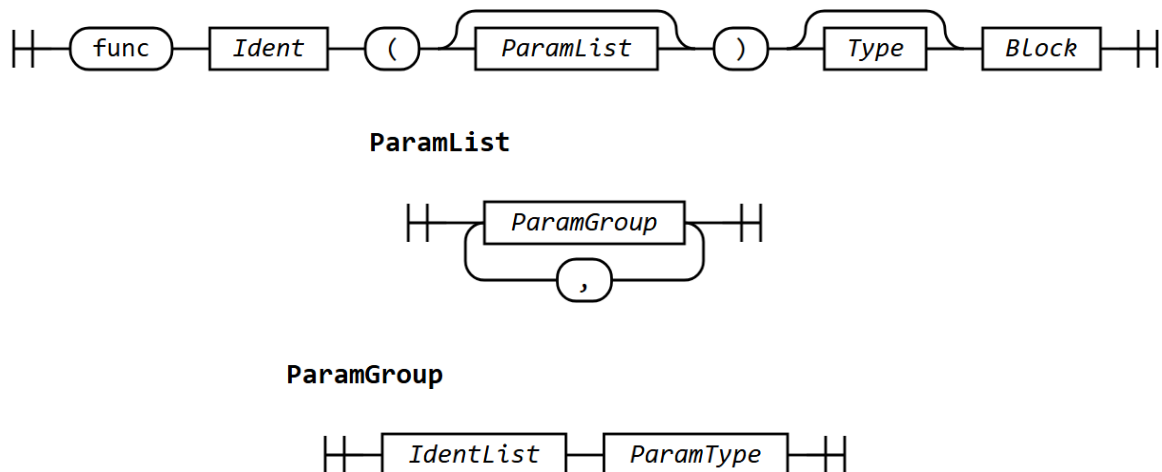


Рисунок 6.4 – Синтаксичні діаграми оголошення функції

Оголошення. Функції оголошуються лише на верхньому рівні; вкладені функції не підтримуються. Ім'я функції унікальне серед імен блоку пакета; перевантаження функцій немає. Функції не є значеннями: їх можна лише викликати.

Виклик. Під час виклику $f(a_1, \dots, a_n)$:

1. Аргументи обчислюються зліва направо. Кількість аргументів має дорівнювати кількості параметрів, а кожен аргумент — бути сумісним за присвоєнням із типом відповідного параметра (СЕ-08).
2. Створюється новий фрейм активації в стеку викликів, який містить параметри, локальні змінні та адресу повернення.
3. Параметрам за значенням (тип T) присвоюються копії значень аргументів; для масивів і структур копіюються всі елементи, тому зміна параметра не впливає на аргумент. Параметрам за посиланням (тип $*T$) передаються адреси змінних (розд. 3.4.3); присвоєння $*p$ змінює саму змінну викликача.
4. Виконується тіло функції до інструкції `return` або до кінця тіла.
5. Фрейм знищується, керування повертається в точку виклику; значенням виклику є значення з `return`.

Виклик функції без результату може бути лише інструкцією (СЕ-15).

Рекурсія. Функція може викликати себе безпосередньо або опосередковано. Кожен рекурсивний виклик створює власний фрейм з окремими екземплярами параметрів і локальних змінних. Максимальна глибина стека викликів визначається реалізацією, але має бути не меншою за 10 000 вкладених викликів; її перевищення спричиняє RE-06.

```
func square(x float) float { return x * x }  
func swap(a, b *int) { t := *a; *a = *b; *b = t }
```

6.5 Області видимості

Мова Krok використовує статичні (лексичні) області видимості. **Блок** — ділянка тексту програми, у якій оголошені імена видимі. Блоки утворюють ієрархію:

1. **Блок пакета** охоплює весь файл. У ньому оголошуються глобальні константи, змінні, типи та функції. Типи й функції видимі в усьому файлі незалежно від порядку оголошень, що дозволяє взаємну рекурсію. Глобальні константи і змінні видимі в тілах усіх функцій, а в ініціалізаторах глобальних оголошень і довжинах масивів — лише після свого оголошення.
2. **Блок функції** містить параметри та інструкції верхнього рівня тіла функції.
3. **Неявний блок інструкції for** містить змінні, оголошені в її ініціалізаційній частині; він охоплює заголовок і тіло циклу.
4. **Явні блоки:** тіла гілок if/else, тіла циклів і вкладені блоки { ... }.

Область видимості локальної змінної чи константи починається одразу після її оголошення (після ініціалізатора) і закінчується в кінці найглибшого блоку, що містить оголошення. Тому у виразі `x := x + 1` ім'я `x` праворуч позначає змінну зовнішнього блоку.

Затінення. Ім'я, оголошене у вкладеному блоці, затінює однойменне ім'я зовнішнього блоку (зокрема глобальне) до кінця вкладеного блоку. Під час розв'язання імені пошук іде від найглибшого блоку до блоку пакета; використовується перше знайдене оголошення.

```

var x int = 1          // глобальна x
func main() {
    x := "local"      // затінює глобальну x
    {
        x := 3.5      // затінює локальну x
        fmt.Println(x) // 3.5
    }
    fmt.Println(x)     // local
}

```

6.6 Час життя та керування пам'яттю

Мова Krok не має динамічного виділення пам'яті та збирання сміття.

Пам'ять розподіляється так:

- **статична область** — глобальні змінні; створюється перед викликом `main` і існує до завершення програми;
- **стек викликів** — фрейми активації функцій з параметрами та локальними змінними, зокрема масивами і структурами, розмір яких відомий на етапі трансляції; фрейм створюється під час виклику і знищується під час повернення; при кожному новому виконанні блоку створюються нові екземпляри його змінних;
- константи не займають пам'яті під час виконання: їхні значення підставляє транслятор.

Рядки незмінні, тому реалізація може розміщувати їхній вміст у керованій нею пам'яті та спільно використовувати однакові значення; це не спостерігається програмою. Висячі посилання неможливі, оскільки вказівники існують лише як параметри і не можуть пережити змінну, на яку посилаються (розд. 3.4.3).

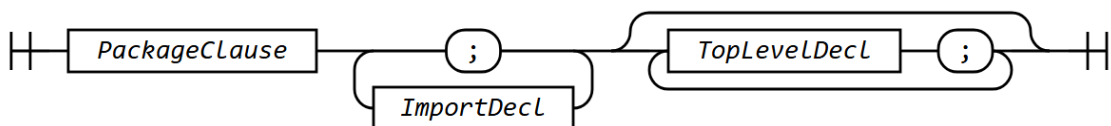
7 Структура програми

7.1 Файл програми, пакет та імпорт

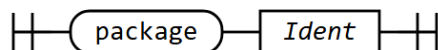
Програма мовою Krok складається з одного файлу, що містить єдиний пакет `main`. Файл починається оголошенням пакета, за яким ідуть нуль або більше оголошень імпорту та нуль або більше оголошень верхнього рівня: змінних, констант, типів і функцій.

```
Program      = PackageClause ';' { ImportDecl ';' } { TopLevelDecl ';' }
PackageClause = package Ident
ImportDecl   = import StringLit
TopLevelDecl = VarDecl | ConstDecl | TypeDecl | FuncDecl
```

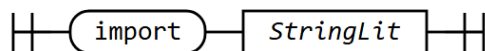
Program



PackageClause



ImportDecl



TopLevelDecl

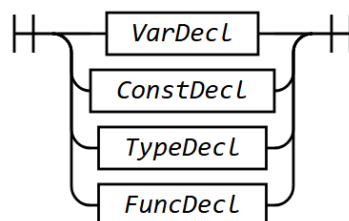


Рисунок 7.1 – Синтаксичні діаграми структури програми

Ім'я пакета, відмінне від `main`, є помилкою SE-13. Розділення програми на кілька файлів чи користувацьких модулів не підтримується.

Оголошення імпорту підключає вбудований пакет. Єдиний доступний пакет — `"fmt"`; імпорт іншого шляху або повторний імпорт `"fmt"` є помилкою SE-14, отже циклічний імпорт неможливий. Доступ до операцій пакета завжди

кваліфікований: `fmt.Println`. Ім'я `fmt` не входить до жодного блоку видимості і не може бути затінене, оскільки є ключовим словом.

```
package main

import "fmt"

func main() {
    fmt.Println("Привіт, Krok!")
}
```

7.2 Функція `main` і модель виконання

Програма повинна містити функцію `main` без параметрів і без результату (СЕ-13). Виконання програми відбувається в такому порядку:

1. Обчислюються значення глобальних констант і ініціалізуються глобальні змінні в порядку їх оголошення в тексті програми. Глобальні змінні без ініціалізатора отримують нульове значення свого типу.
2. Викликається функція `main`.
3. Після повернення з `main` програма завершується з кодом завершення 0.

Виконання програми **завершується нормально** тоді і тільки тоді, коли нормально завершується виконання кожної її інструкції, тобто коли під час виконання не виявлено жодної помилки. Якщо виконання інструкції завершується помилкою (розд. 7.4), програма **завершується аварійно**: виконання негайно припиняється з кодом завершення 2, а повідомлення про помилку записується в стандартний потік помилок. Дані, виведені до помилки, зберігаються.

7.3 Помилки трансляції

Транслятор повинен виявити всі наведені нижче помилки до початку виконання програми; програма з помилкою трансляції не виконується.

Таблиця 7.1 – Помилки трансляції

Код	Помилка
-----	---------

Код	Помилка
CE-01	лексична помилка: недопустимий символ, ведучий нуль у цілому літералі, неправильна експонента, незакритий рядковий літерал, невідома escape-послідовність
CE-02	синтаксична помилка: послідовність токенів не відповідає граматиці
CE-03	використання оголошеного імені
CE-04	повторне оголошення імені в тому самому блоці
CE-05	невідповідність типів: операнди оператора, присвоєння, ініціалізація, умова if/for, аргументи Print/Scan, кількість виразів у var
CE-06	недопустиме явне приведення типів
CE-07	присвоєння, ++/--, & чи введення для виразу, що не є l-значенням (константи, результату виклику тощо)
CE-08	невідповідність кількості або типів аргументів виклику; виклик імені, що не є функцією чи типом
CE-09	return без виразу у функції з результатом, з виразом — у функції без результату; відсутність return на деякому шляху виконання
CE-10	значення літерала поза множиною значень типу; помилка обчислення константного виразу
CE-11	довжина масиву не є додатною константою типу int; зайві елементи в літералі масиву; константний індекс поза межами
CE-12	повторне або неіснуюче поле структури; рекурсивне включення структури в себе
CE-13	ім'я пакета відмінне від main; відсутність функції main або неправильна її сигнатура
CE-14	використання fmt без імпорту; імпорт шляху, відмінного від "fmt"; повторний імпорт
CE-15	використання виклику функції без результату як значення
CE-16	ініціалізатор глобальної змінної не є константним виразом
CE-17	ліва частина := не є ідентифікатором; := в інструкції наступного кроку for; інструкція-вираз не є викликом функції

7.4 Помилки часу виконання

У разі помилки часу виконання програма припиняє роботу з кодом завершення 2 і записує в стандартний потік помилок повідомлення у форматі `runtime error RE-NN at рядок:стовпчик: опис`.

Таблиця 7.2 – Помилки часу виконання

Код	Помилка	Причина
RE-01	ділення на нуль	<code>a / 0</code> , <code>a % 0</code> для <code>int</code> ; <code>x / 0.0</code> для <code>float</code>
RE-02	переповнення цілого	результат <code>+</code> <code>-</code> <code>*</code> <code>**</code> <code>++</code> <code>--</code> чи унарного <code>-</code> поза межами <code>int</code> (зокрема <code>-9223372036854775807 - 2</code>)
RE-03	переповнення дійсного	результат дійсної операції — нескінченність або NaN (<code>1e308 * 10</code> , <code>(-8.0) ** 0.5</code>)
RE-04	вихід за межі масиву	індекс <code>i < 0</code> або <code>i ≥ len(a)</code>
RE-05	від'ємний показник	<code>a ** b</code> для <code>int</code> з <code>b < 0</code>
RE-06	переповнення стека	перевищено максимальну глибину викликів (нескінченна рекурсія)
RE-07	некоректні вхідні дані	слово, прочитане <code>fmt.Scan</code> , не є значенням типу змінної
RE-08	кінець вхідного потоку	потік введення закінчився під час виконання <code>fmt.Scan</code>
RE-09	недопустиме приведення	<code>int(x)</code> , де значення <code>x</code> після відкидання дробової частини поза межами <code>int</code>

8 Властивості граматик

8.1 Кількісні характеристики

Граматика мови Krok представлено у формі $\Gamma = (N, T, P, S)$ (розд. 1.5.2). Кількісні характеристики обох її частин обчислює програма `tools/check_grammar.py`; їх наведено в табл. 8.1.

Таблиця 8.1 – Кількісні характеристики граматик мови Krok

Характеристика	Синтаксична граматика	Лексична граматика
Аксіома S	Program	SourceText, Alphabet
Нетермінали N	64	33
Термінали T	51 + 4 класи токенів	123 символів + 3 спец. послідовності
Правила РБНФ P	64	33
Клас мови	контекстно вільна (тип 2), LL(1)	регулярна (тип 3); ДСА

Чотири правила (BoolLit, RelOp, AssignOp, IncDecOp) належать обома частинам граматик, тому загальна кількість правил у файлі `grammar/krok.ebnf` становить 93.

8.2 Практичні трансформації: факторизація та заміна рекурсії ітерацією

Граматика виразів побудовано з природної, але ліворекурсивної форми, яку зазвичай використовують для опису пріоритетів операторів, методом заміни рекурсії на ітерацію (розд. 1.3.5 у [1]). Вихідні правила:

```
AddExpr = MulExpr | AddExpr '+' MulExpr | AddExpr '-' MulExpr
MulExpr = UnaryExpr | MulExpr '*' UnaryExpr | MulExpr '/' UnaryExpr
          | MulExpr '%' UnaryExpr
```

Правила виду $U = A \mid U \times \mid U y$ спочатку факторизуються до $U = A \mid U (\times \mid y)$, а потім ліва рекурсія замінюється ітерацією: $U = A \{ \times \mid y \}$. Після застосування цих кроків і винесення груп операторів в окремі правила AddOp та MulOp отримуємо форму, використану в цій специфікації:

```
AddExpr = MulExpr { AddOp MulExpr }
MulExpr = UnaryExpr { MulOp UnaryExpr }
```

Обидві форми породжують одну й ту саму мову і задають ту саму (ліву) асоціативність операторів, але друга не має лівої рекурсії, а тому придатна для аналізу методом рекурсивного спуску. Аналогічно перетворено правила OrExpr, AndExpr і PrimaryExpr, а також списки IdentList, ExpressionList, ParamList та ArgumentList (форма $X \{ ' , ' X \}$).

Праву асоціативність оператора ****** реалізовано правою рекурсією `PowerExpr = PrimaryExpr [ '**' UnaryExpr ]`, яка для низхідного аналізу перешкодою не є.

Факторизацію застосовано й до інструкцій: альтернативи «присвоєння», «оголошення з ініціалізацією», «інкремент» і «виклик функції» мають спільний префікс — вираз, тому їх об'єднано в одне правило `SimpleStmt = Expression [ AssignOp Expression | IncDecOp ]`, що усуває потребу у поверненнях під час розбору.

Списки інструкцій і полів записано правою рекурсією (`StatementList/StatementTail`, `FieldList/FieldTail`), оскільки роздільник ; може бути як явним, так і вставленим, а останню крапку з комою перед } дозволено пропускати.

8.3 Стандартні трансформації граматик

Теорія формальних мов розглядає чотири стандартні трансформації контекстно вільних граматик — видалення ϵ -продукцій, марних символів, ланцюгових продукцій і циклів — та дві нормальні форми (Хомського і Грейбах). Стан граматика мови Krok щодо кожної з них наведено в табл. 8.2; числові значення обчислює програма `tools/check_grammar.py --transform`.

Таблиця 8.2 – Стандартні трансформації та стан граматика мови Krok

Трансформація	Стан граматика мови Krok
Видалення ϵ -продукцій	правил виду $A \rightarrow \epsilon$ немає: порожній ланцюжок породжують лише метасимволи [] та { }. Анульовними є 5 нетермінали: <code>ElementTail</code> , <code>FieldList</code> , <code>FieldTail</code> , <code>StatementList</code> , <code>StatementTail</code> . Перевірка вимагає, щоб тіло [] чи { } само не було анульовним
Видалення марних символів	не потрібне: марних символів немає

Трансформація	Стан граматики мови Krok
Видалення ланцюгових продукцій	60 ланцюгових продукцій збережено свідомо: вони або класифікують конструкції (Statement, Operand, Type, Token), або іменують рівні пріоритету (Expression = OrExpr). Для LL(1)-аналізу вони нешкідливі — кожній відповідає один виклик процедури рекурсивного спуску
Видалення циклів	не потрібне: циклів $A \Rightarrow^+ A$ немає
Факторизація	застосовано до SimpleStmt (спільний префікс — вираз) та до груп операторів AddOp, MulOp
Заміна рекурсії ітерацією	застосовано до всіх рівнів виразів і до списків $X \{ ', ' X \}$ — див. розд. 8.2
Нормальні форми Хомського та Грейбах	не застосовуються: вони потрібні алгоритмам загального КВ-розбору (наприклад, СΥΚ), тоді як для рекурсивного спуску достатньо відсутності лівої рекурсії та факторизації; крім того, бінарна нормальна форма несумісна з метасимволами РБНФ

Отже, граматика не потребує жодної з обов'язкових трансформацій, а ланцюгові продукції збережено свідомо: вони не заважають ані LL(1)-аналізу, ані генеруванню синтаксичних діаграм, натомість роблять граматику придатною для читання людиною, що для специфікації є головним.

8.4 Автоматична перевірка граматики

Властивості граматики перевірено автоматично програмою tools/check_grammar.py, яка обчислює множини FIRST і FOLLOW, будує граф лівих виведень і перевіряє умови LL(1) для кожної альтернативи, необов'язкової частини та повторення. Результати перевірки наведено в табл. 8.3. Правильність самої перевірки підтверджено на навмисно зіпсованих варіантах граматики (з лівою рекурсією, конфліктом FIRST/FIRST, «висячим else» та недосяжним правилом): у кожному випадку дефект виявлено.

Таблиця 8.3 – Результати автоматичної перевірки граматики

№	Властивість граматики	Результат
1	Усі нетермінали, що використовуються, визначені	виконується
2	Граматика контекстно-вільна: у лівій частині кожного правила один нетермінал	виконується
3	Недосяжні символи відсутні	виконується
4	Непродуктивні символи відсутні	виконується
5	Ліва рекурсія (безпосередня та опосередкована) відсутня	виконується
6	Цикли $A \Rightarrow^+ A$ (ланцюгові) відсутні	виконується
7	Синтаксична граматика належить до класу LL(1), отже однозначна	виконується
8	Усі термінали складені із символів алфавіту	виконується
9	Синтаксична граматика оперує токенами, а не символами	виконується

Коментарі до пунктів контрольного списку:

- **Контекстна вільність.** Ліва частина кожного правила містить рівно один нетермінал, отже граматики належить до типу 2 за класифікацією Хомського.
- **Марні символи.** Кожен нетермінал досяжний від однієї з аксіом (Program, SourceText або Alphabet) і виводить термінальний ланцюжок, тобто є корисним.
- **Ліва рекурсія.** Відсутня — див. розд. 8.2.
- **Однозначність.** Кожна LL(1)-граматика однозначна. Проблему «висячого else» усунено обов'язковими фігурними дужками, неоднозначність складених літералів у заголовках if/for (відому в мові Go) — вимогою явного типу [N]T у літералі масиву.
- **Алфавіт.** Усі термінали складаються із символів, визначених правилами Letter, Digit, SpecialChar, WhiteSpace та EndOfLine.
- **Розмежування граматик.** Синтаксична граматика звертається до лексичної лише через токени Ident, IntLit, FloatLit, StringLit та групи токенів BoolLit, RelOp, AssignOp, IncDecOp.
- **Літерали.** Граматика охоплює всі форми цілих (без ведучих нулів), дійсних (з дробовою частиною, з експонентою, з пропущеною цілою або дробовою частиною), рядкових (порожній рядок, escape-послідовності) і логічних літералів; знак числа — унарний оператор.
- **Пріоритет, асоціативність і арність.** Пріоритет забезпечено стратифікацією, асоціативність — повторенням (ліва) та правою рекурсією (права, **). Унарні оператори (UnaryOp, Dereference, AddrExpr) і бінарні (AddOp, MulOp, RelOp, &&, ||, **) описано окремими правилами.
- **Оголошення змінних і констант** розрізняються ключовим словом (var / const); константа оголошує рівно одне ім'я й обов'язково має ініціалізатор.

- **Еквівалентність РБНФ і діаграм.** Діаграми згенеровано з того самого файлу граматики; програма побудови документа перевіряє, що кожне правило подано в розд. 2–7 рівно один раз.

8.5 Граматика та контекстні умови

Граматикою виражено лише структуру програми. Контекстні умови перевіряються семантичним аналізатором; розподіл наведено в табл. 8.4.

Таблиця 8.4 – Інформація, виражена граматикою, та контекстні умови

Виражено граматикою	Контекстні умови (семантичний аналіз)
пріоритет і асоціативність операторів	сумісність типів операндів і результатів
обов'язкові фігурні дужки блоків	оголошеність імен і відсутність повторних оголошень
struct лише в оголошеннях типів	ліва частина = є l-значенням, ліва частина := — ідентифікатором
*T лише в типах параметрів	інструкція-вираз є викликом функції
аргументи Scan мають форму &вираз	операнд & є адресованим виразом
константа має рівно одне ім'я та ініціалізатор	ініціалізатор константи є константним виразом
розіменування лише ідентифікатора	розіменовуваний ідентифікатор є параметром-вказівником
трикомпонентний for має всі три частини	умови if/for мають тип bool; post-частина не є :=
оператори порівняння неасоціативні	довжина масиву — додатна константа типу int
типи та функції лише на верхньому рівні	наявність return у всіх гілках, сигнатура main, import "fmt"

9 Приклади програм

Усі наведені нижче програми (крім прикладів із розд. 9.10) є лексично і синтаксично правильними, що перевірено програмою `tools/krok_parse.py` за граматикою з розд. 10.

9.1 Базовий приклад

Базовий приклад містить принаймні один екземпляр кожної лексеми з таблиці токенів (табл. 2.2); повноту покриття перевірено автоматично командою

```
python tools/krok_parse.py --coverage examples/base_example.krok.
```

```
1 // Базовий приклад програми мовою Krok:
2 // містить приклади всіх лексем із таблиці токенів.
3 package main
4
5 import "fmt"
6
7 const Size int = 5 // іменована константа з явним типом
8 const Eps = 1.5e-3 // тип float визначається за літералом
9 const Title = "Krok:\t\"крок\" \\ 2026"
10
11 type Celsius float // користувацький скалярний тип
12
13 type Point struct {
14     x, y float
15     label string
16 }
17
18 var total int // глобальна змінна, початкове значення 0
19 var ready bool = false // глобальна змінна з ініціалізацією
20
21 // Рекурсивне обчислення факторіала.
22 func factorial(n int) int {
23     if n <= 1 {
24         return 1
25     }
26     return n * factorial(n-1)
27 }
28
29 // Передача параметрів за посиланням.
30 func swap(a, b *int) {
31     t := *a
32     *a = *b
33     *b = t
34 }
35
36 // Масив передається за значенням (копіюється).
37 func sum(v [Size]int) int {
38     s := 0
39     for i := 0; i < len(v); i++ {
```

```

40     s = s + v[i]
41 }
42 return s
43 }
44
45 func distance(p Point) float {
46     return (p.x**2 + p.y**2) ** .5
47 }
48
49 func main() {
50     var n, m int
51     fmt.Scan(&n, &m)
52     swap(&n, &m)
53     fmt.Println("n =", n, "m =", m, "n! =", factorial(n%10))
54
55     var data [Size]int = [Size]int{3, -1, 4, 1, +5}
56     var grid [2][3]float
57     grid[1][2] = 2.
58     total = sum(data)
59
60     var t Celsius = 36.6
61     t = t + Celsius(n)
62     x := float(total)/2 - 7*3 // int неявно розширюється до float
63     k := int(-x)
64
65     var p Point
66     p.x = 3.0
67     p.y = 4E0
68     p.label = ""
69     d := distance(p)
70
71     ok := true
72     ready = d >= 5.0 && !(k == 0) || total != 12
73     for k > 10 {
74         k--
75     }
76     for i := 0; i < 3; i++ {
77         if grid[1][2] > float(i) {
78             fmt.Print(i, " ")
79         } else if i == 2 {
80             fmt.Print("last")
81         } else {
82             fmt.Print("-")
83         }
84     }
85     fmt.Println()
86     fmt.Print(Title, "\n")
87     fmt.Println(t, d, ready, ok, Eps, x < -1.0, 2**3**2, k)
88 }

```

Для вхідних даних 4 7 програма виводить:

```

n = 7 m = 4 n! = 5040
0 1 last
Krok: "крок" \ 2026
43.6 5.0 true true 0.0015 true 512 10

```

9.2 Типи та приведення

```
1 // Типи, неявне та явне приведення
2 package main
3
4 import "fmt"
5
6 func main() {
7     var i int = 7
8     var f float = 2 // int-літерал неявно розширюється до float
9     a := i / 2      // int / int → int: 3 (відкидання дробової частини)
10    b := i / 2.0     // int / float → float: 3.5
11    c := float(i) / 2 // явне приведення: 3.5
12    d := int(-3.9)   // відкидання дробової частини в бік нуля: -3
13    e := i % 4       // остача: 3
14    s := "Kr" + "ok" // конкатенація рядків
15    fmt.Println(a, b, c, d, e, s, f ** 2, len(s))
16 }
```

Результат виконання: 3 3.5 3.5 -3 3 Krok 4.0 4.

9.3 Розгалуження

```
1 // Розгалуження: if / else if / else
2 package main
3
4 import "fmt"
5
6 func main() {
7     var t float
8     fmt.Scan(&t)
9     if t < 0 {
10         fmt.Println("мороз")
11     } else if t < 15 {
12         fmt.Println("прохолодно")
13     } else if t <= 25 && t != 20 {
14         fmt.Println("комфортно")
15     } else {
16         fmt.Println("спекотно або рівно 20")
17     }
18 }
```

Для вхідного значення 22 програма виводить комфортно, для 20 — спекотно або рівно 20, для -3.5 — мороз.

9.4 Цикли

```
1 // Цикли: з передумовою та з лічильником (трикомпонентний)
2 package main
3
4 import "fmt"
5
6 func main() {
7     var n int
8     fmt.Scan(&n)
9 }
```

```

10 // цикл з передумовою: сума цифр числа
11 digits := 0
12 for n > 0 {
13     digits = digits + n%10
14     n = n / 10
15 }
16 fmt.Println("сума цифр:", digits)
17
18 // вкладені цикли з лічильником: таблиця множення 3x3
19 for i := 1; i <= 3; i++ {
20     for j := 1; j <= 3; j++ {
21         fmt.Print(i*j, "\t")
22     }
23     fmt.Println()
24 }
25
26 // лічильник можна змінювати в тілі циклу
27 for k := 10; k > 0; k-- {
28     k = k - 2
29     fmt.Print(k, " ")
30 }
31 }

```

Для вхідного значення 2026 програма виводить:

```

сума цифр: 10
1   2   3
2   4   6
3   6   9
8 5 2 -1

```

9.5 Функції та рекурсія

```

1 // Функції та рекурсія
2 package main
3
4 import "fmt"
5
6 func fib(n int) int {
7     if n < 2 {
8         return n
9     }
10    return fib(n-1) + fib(n-2)
11 }
12
13 func gcd(a, b int) int {
14     if b == 0 {
15         return a
16     }
17    return gcd(b, a%b)
18 }
19
20 func power(base float, e int) float {
21     if e == 0 {
22         return 1
23     } else if e < 0 {
24         return 1 / power(base, -e)
25     }

```

```

26     return base * power(base, e-1)
27 }
28
29 func main() {
30     fmt.Println(fib(10), gcd(84, 36), power(2, -3), 2.0**-3)
31 }

```

Результат виконання: 55 12 0.125 0.125.

9.6 Передача параметрів за посиланням

```

1 // Передача параметрів за посиланням: сортування масиву
2 package main
3
4 import "fmt"
5
6 const N = 6
7
8 func swap(a, b *int) {
9     t := *a
10    *a = *b
11    *b = t
12 }
13
14 func sort(a *[N]int) {
15     for i := 0; i < N-1; i++ {
16         for j := 0; j < N-1-i; j++ {
17             if (*a)[j] > (*a)[j+1] {
18                 swap(&(*a)[j], &(*a)[j+1])
19             }
20         }
21     }
22 }
23
24 func main() {
25     var v [N]int
26     for i := 0; i < N; i++ {
27         fmt.Scan(&v[i])
28     }
29     sort(&v)
30     fmt.Println(v[0], v[1], v[2], v[3], v[4], v[5])
31 }

```

Для вхідних даних 5 2 9 1 7 3 програма виводить 1 2 3 5 7 9.

9.7 Масиви та структури

```

1 // Масиви (статична та динамічна ініціалізація) і структури
2 package main
3
4 import "fmt"
5
6 type Vec struct {
7     x, y float
8 }
9
10 type Segment struct {

```

```

11     from, to Vec
12     name      string
13 }
14
15 func length(s Segment) float {
16     dx := s.to.x - s.from.x
17     dy := s.to.y - s.from.y
18     return (dx*dx + dy*dy) ** 0.5
19 }
20
21 func main() {
22     m := [2][2]int{
23         {1, 2},
24         {3, 4},
25     }
26     var sq [5]int // нульові значення
27     for i := 0; i < len(sq); i++ {
28         sq[i] = i * i // динамічна ініціалізація
29     }
30
31     var s Segment
32     s.name = "AB"
33     s.to.x = 3
34     s.to.y = 4
35     c := s // поелементне копіювання
36     c.name = "copy"
37
38     fmt.Println(m[1][0], sq[4], length(s), s.name, c.name)
39 }

```

Результат виконання: 3 16 5.0 AB copy.

9.8 Области видимості та затінення

```
1 // Области видимості та затінення
2 package main
3
4 import "fmt"
5
6 var x int = 1 // глобальна
7
8 func show() {
9     fmt.Println("global x =", x) // глобальна x: 1
10 }
11
12 func main() {
13     fmt.Println(x) // 1
14     x := "local"    // затінює глобальну x у тілі main
15     {
16         x := 3.5 // затінює локальну x у вкладеному блоці
17         fmt.Println(x)
18     }
19     fmt.Println(x) // local
20     for i := 0; i < 2; i++ {
21         x := i // нова змінна на кожне виконання тіла
22         fmt.Println(x)
23     }
24     show()
25 }
```

Результат виконання:

```
1
3.5
local
0
1
global x = 1
```

9.9 Помилки часу виконання

```
1 // Помилки часу виконання
2 package main
3
4 import "fmt"
5
6 func depth(n int) int {
7     return depth(n+1) + 1 // RE-06: переповнення стека
8 }
9
10 func main() {
11     var a [3]int
12     var i, z int
13     fmt.Scan(&i, &z)
14     fmt.Println(a[i]) // RE-04 при i < 0 або i > 2
15     fmt.Println(10 / z) // RE-01 при z == 0
16     big := 9223372036854775807
17     fmt.Println(big + 1) // RE-02: переповнення int
18     fmt.Println(depth(0))
19 }
```

Для вхідних даних 5 1 виконання припиняється на першому ж виведенні: runtime error RE-04 at 14:15: index 5 out of range [0, 2]. Для 1 0 програма виводить 0 і завершується з RE-01 у рядку 15.

9.10 Некоректні програми

Таблиця 9.1 – Приклади некоректних фрагментів програм

Фрагмент	Помилка
} i else { у різних рядках	CE-02: після } вставлено ;, очікується }, отримано else
if x > 0 x = 2	CE-02: очікується {
x := 007	CE-01: цілий літерал із ведучим нулем
type Local int у тілі функції	CE-02: оголошення типу лише на верхньому рівні
var p *int	CE-02: вказівник може бути лише типом параметра
a := 1 < 2 < 3	CE-02: оператори порівняння неасоціативні
a = b = 3	CE-02: присвоєння є інструкцією, а не виразом
fmt.Scan(n)	CE-02: аргумент Scan має форму &вираз
var n int = 2.5	CE-05: неявне звуження float → int заборонене
if n { } для n int	CE-05: умова має тип int, а не bool
const N = 3 i N = 4	CE-07: присвоєння константі
x := 1 i x := 2 в одному блоці	CE-04: повторне оголошення
func f() int { if a { return 1 } }	CE-09: немає return на шляху, коли умова хибна
var a [0]int	CE-11: довжина масиву має бути додатною
fmt.Println(int("5"))	CE-06: приведення string → int недопустиме

Перші вісім фрагментів відхиляються вже синтаксичним аналізатором; їх повні тексти містяться в каталозі examples/invalid і перевіряються командою python tools/krok_parse.py --invalid examples/invalid/*.krok.

Перелік посилань

1. Стативка Ю. І. Формальні мови: Основні концепти і представлення [Текст]: навч. посіб. / Ю. І. Стативка. — Київ: КПІ ім. Ігоря Сікорського, 2023. — 87 с.
2. Медведєва В. М. Транслятори: лексичний та синтаксичний аналізатори [Текст]: навч. посіб. / В. М. Медведєва, В. А. Третяк. — К.: НТУУ «КПІ», 2012. — 148 с.
3. Aho A., Lam M., Sethi R., Ullman J. Compilers: Principles, Techniques, and Tools. 2nd ed. Addison-Wesley, 2006. 1040 p.
4. The Go Programming Language Specification. URL: <https://go.dev/ref/spec>
5. Effective Go. URL: https://go.dev/doc/effective_go
6. EBNF Visualizer. URL: <http://dotnet.jku.at/applications/visualizer/>

10 Повна граматика мови

Повну граматика мови Krok наведено в порядку від кореневого нетермінала Program до низькорівневих лексичних конструкцій Letter та Digit. Текст збігається з файлом grammar/krok.ebnf, з якого згенеровано всі синтаксичні діаграми цієї специфікації.

```
(* =====
Krok – повна граматика мови (РБНФ)
Синтаксична аксіома: Program
Лексична аксіома:   SourceText
Алфавіт:            Alphabet
===== *)

(* § Структура програми *)
Program      = PackageClause ';' { ImportDecl ';' } { TopLevelDecl ';' }
PackageClause = package Ident
ImportDecl   = import StringLit
TopLevelDecl = VarDecl | ConstDecl | TypeDecl | FuncDecl

(* § Оголошення *)
VarDecl      = var IdentList
              ( Type [ '=' ExpressionList ] | '=' ExpressionList )
ConstDecl    = const Ident [ Type ] '=' Expression
TypeDecl     = type Ident ( Type | StructType )
StructType   = struct '{' FieldList '}'
FieldList    = [ FieldDecl FieldTail ]
FieldTail    = [ ';' FieldList ]
FieldDecl    = IdentList Type
IdentList    = Ident { ',' Ident }

(* § Типи *)
Type         = BasicType | TypeName | ArrayType
BasicType    = int | float | bool | string
TypeName     = Ident
ArrayType    = '[' Expression ']' Type

(* § Функції *)
FuncDecl     = func Ident '(' [ ParamList ] ')' [ Type ] Block
ParamList    = ParamGroup { ',' ParamGroup }
ParamGroup   = IdentList ParamType
ParamType    = [ '*' ] Type

(* § Інструкції *)
Block        = '{' StatementList '}'
StatementList = [ Statement StatementTail ]
StatementTail = [ ';' StatementList ]
Statement    = VarDecl | ConstDecl | IfStmt | ForStmt | ReturnStmt
              | IoStmt | Block | SimpleStmt
SimpleStmt   = Expression [ AssignOp Expression | IncDecOp ]
AssignOp     = '=' | ':='
IncDecOp     = '++' | '--'
IfStmt       = if Expression Block [ else ( IfStmt | Block ) ]
ForStmt      = for ForHeader Block
ForHeader    = SimpleStmt [ ';' Expression ';' SimpleStmt ]
ReturnStmt   = return [ Expression ]
IoStmt       = fmt '.' ( OutputCall | InputCall )
```

```

OutputCall    = ( 'Print' | 'Println' ) '(' [ ExpressionList ] ')'
InputCall     = 'Scan' '(' AddrExpr { ',' AddrExpr } ')'

(* § Вирази *)
ExpressionList = Expression { ',' Expression }
Expression     = OrExpr
OrExpr         = AndExpr { '||' AndExpr }
AndExpr        = RelExpr { '&&' RelExpr }
RelExpr        = AddExpr [ RelOp AddExpr ]
AddExpr        = MulExpr { AddOp MulExpr }
MulExpr        = UnaryExpr { MulOp UnaryExpr }
UnaryExpr      = UnaryOp UnaryExpr | PowerExpr
PowerExpr      = PrimaryExpr [ '**' UnaryExpr ]
PrimaryExpr    = Operand { Selector | Index | Arguments }
Selector       = '.' Ident
Index          = '[' Expression ']'
Arguments      = '(' [ ArgumentList ] ')'
ArgumentList   = Argument { ',' Argument }
Argument       = AddrExpr | Expression
AddrExpr       = '&' PrimaryExpr
Operand        = Literal | Ident | Dereference | Conversion | LenCall
               | ArrayLit | '(' Expression ')'
Dereference    = '*' Ident
Conversion     = BasicType '(' Expression ')'
LenCall        = len '(' Expression ')'
ArrayLit       = ArrayType LiteralValue
LiteralValue   = '{' [ Element ElementTail ] '}'
ElementTail    = [ ',' [ Element ElementTail ] ]
Element        = Expression | LiteralValue
Literal        = IntLit | FloatLit | StringLit | BoolLit
BoolLit        = true | false
RelOp          = '==' | '!=' | '<' | '<=' | '>' | '>='
AddOp          = '+' | '-'
MulOp          = '*' | '/' | '%'
UnaryOp        = '-' | '+' | '!'

(* =====
   Лексична граматика
   ===== *)

(* § Лексеми *)
SourceText    = { Lexeme }
Lexeme        = Token | Comment | WhiteSpace | EndOfLine
Token         = Ident | Keyword | IntLit | FloatLit | StringLit | BoolLit
               | Operator | Delimiter
Keyword       = package | import | func | var | const | type | struct
               | if | else | for | return | int | float | bool | string
               | len | fmt | 'Print' | 'Println' | 'Scan'
Operator      = ArithOp | RelOp | LogicOp | AssignOp | IncDecOp | '&'
ArithOp       = '+' | '-' | '*' | '/' | '%' | '**'
LogicOp       = '&&' | '||' | '!'
Delimiter     = '(' | ')' | '[' | ']' | '{' | '}' | ',' | ';' | '.'

(* § Ідентифікатори та літерали *)
Ident         = Letter { Letter | Digit }
IntLit        = '0' | NonZeroDigit { Digit }
FloatLit      = Digits '.' [ Digits ] [ Exponent ]
               | Digits Exponent
               | '.' Digits [ Exponent ]
Digits        = Digit { Digit }
Exponent      = ( 'e' | 'E' ) [ '+' | '-' ] Digits
StringLit     = '"' { StringChar } '"'
StringChar    = OrdinaryChar | EscapeSeq

```

```

EscapeSeq      = '\' ( 'n' | 't' | 'r' | '"' | '\'' )
OrdinaryChar    = ? будь-який символ Unicode, крім '"', '\', '\n' та '\r' ?
Comment         = '//' { CommentChar }
CommentChar     = ? будь-який символ Unicode, крім '\n' та '\r' ?

(* § Алфавіт *)
Alphabet        = Letter | Digit | SpecialChar | WhiteSpace | EndOfLine | OtherChar
Letter          = LowerLetter | UpperLetter | '_'
LowerLetter     = 'a' | 'b' | 'c' | 'd' | 'e' | 'f' | 'g' | 'h' | 'i'
                | 'j' | 'k' | 'l' | 'm' | 'n' | 'o' | 'p' | 'q' | 'r'
                | 's' | 't' | 'u' | 'v' | 'w' | 'x' | 'y' | 'z'
UpperLetter    = 'A' | 'B' | 'C' | 'D' | 'E' | 'F' | 'G' | 'H' | 'I'
                | 'J' | 'K' | 'L' | 'M' | 'N' | 'O' | 'P' | 'Q' | 'R'
                | 'S' | 'T' | 'U' | 'V' | 'W' | 'X' | 'Y' | 'Z'
Digit          = '0' | NonZeroDigit
NonZeroDigit    = '1' | '2' | '3' | '4' | '5' | '6' | '7' | '8' | '9'
SpecialChar     = '+' | '-' | '*' | '/' | '%' | '=' | '!' | '<' | '>'
                | '&' | '|' | ':' | ';' | ',' | '.' | '(' | ')' | '['
                | ']' | '{' | '}' | '"' | '\'
WhiteSpace     = ' ' | '\t'
EndOfLine      = '\n' | '\r' '\n'
OtherChar       = ? інший символ Unicode (лише в StringLit та Comment) ?

```